Time Series and Sequential Data

Volker Tresp

Summer 2023

Time Series Modelling

- I have to predict the total energy consumption of a city for tomorrow, based on certain inputs (weather forecast: temperature, precipitation, wind; then: working day/holiday, ...)
- It would help to also consider the energy consumption of today and maybe of yesterday as inputs
- Added complexity: If my goal is to predict the energy consumption two days in the future, my own prediction for tomorrow becomes an input for the prediction for two days in the future
- In time series modelling, outputs, and often also the inputs, are real numbers

DAX Performance-Index

01.06.07 17:45 Uhr • 7.987,85 +1,33 % [+104,81]	Enthaltene Werte: **30** Tages-Vol.: **9,12 Mrd.**	Typ: **Index** Börse: XETRA

Sequence Modelling

- Sequence classification: The input is a sentence, i.e., a sequence of words; the output classifies the sentiment of the sentence
- Encoder-decoder modelling: The input is a sentence, i.e., a sequence of words, in English; the output is the sentence translated into German
- In sequence modelling, inputs and outputs are typically discrete

I. Time Series Modelling: NARX Models

Neural Networks for Time-Series Modelling

- Let $y_t, t = 1, 2, \ldots$ be the time-discrete time-series of interest (example: DAX)
- Let $x_t, t = 1, 2, \ldots$ denote a second time-series, that contains information on y_t (Example: Dow Jones)
- For simplicity, we assume that both y_t and x_t are scalars. The goal is the prediction of the next value of the time-series
- We assume a system of the form

$$y_t = f(y_{t-1}, \ldots, y_{t-T}, x_{t-1}, \ldots, x_{t-T}) + \epsilon_t$$

with i.i.d. random numbers $\epsilon_t, t = 1, 2, \ldots$ which model unknown disturbances

Neural Networks for Time-Series Modelling (cont'd)

- We approximate the function, using a neural network,

$$f(y_{t-1}, \ldots, y_{t-T}, x_{t-1}, \ldots, x_{t-T})$$

$$\approx f_{\mathbf{w},V}(y_{t-1}, \ldots, y_{t-T}, x_{t-1}, \ldots, x_{t-T})$$

- A reasonable cost function is

$$\text{cost}(\mathbf{w}, V) = \sum_{t=1}^{N} (y_t - f_{\mathbf{w},V}(y_{t-1}, \ldots, y_{t-T}, x_{t-1}, \ldots, x_{t-T}))^2$$

Neural Networks for Time-Series Modelling (cont'd)

- It is important to note, that the neural network can be trained as before with simple back propagation *if in training all* y_t *and all* x_t *are known*!
- This model is called a NARX model: **N**onlinear **A**uto **R**egressive Model with e**x**ternal inputs. Another name: TDNN (time-delay neural network)
- Note the "convolutional" idea in TDNNs: the same neural network is applied in all time instances

Prediction

- For single step prediction, we use

$$\widehat{y}_t = f(y_{t-1}, \ldots, y_{t-T}, x_{t-1}, \ldots, x_{t-T})$$

Multiple-Step Prediction based on Multiple Step Prediction

- We can also train a model to predict τ time steps into the future; the prediction then becomes

$$\widehat{y}_{t+\tau} = f^{\tau}(y_{t-1}, \ldots, y_{t-T}, x_{t-1}, \ldots, x_{t-T})$$

- This is done in system simulation: the prediction based on detailed system models might be computationally very expensive and cannot be done online; the idea is to train a neural network predictive model off-line and then use that one online instead of an expensive simulation

Mutiple-Step Prediction based on Single-Step Prediction

- Why not just iterate the single-step prediction? One issue is that my prediction is uncertain, so I should consider that uncertainty; second: I do not have future inputs!
- One way is simulation. Let's assume that for x_t we have a deterministic model

$$x_t = f^x(y_{t-1}, \ldots, y_{t-T}, x_{t-1}, \ldots, x_{t-T})$$

  and for y_t we have the model as before,

$$y_t = f(y_{t-1}, \ldots, y_{t-T}, x_{t-1}, \ldots, x_{t-T}) + \epsilon_t$$

- Using both we can generate samples for the future; for the noise I might assume a Gaussian distribution $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$
- For multiple-step prediction, we can simulate (i.e., sample) for the desired number of time steps in the future (Monte-Carlo simulation) repeatedly and can derive estimated means, variances, and covariances

Residual Modeling

- We have

$$\widehat{y}_t = y_{t-1} + f(y_{t-1}, \ldots, y_{t-T}, x_{t-1}, \ldots, x_{t-T})$$

- Realize the similarity to ResNet

II. Sequence Modelling

A: One-hot Encoding

- Let's assume that $x_t = i$ stands for the fact that the word at position t has word index $i \in \{1, 2, ..., N_w\}$, where N_w is the number of words in the vocabulary
- A: One-hot encoding: if x_t is an input to a neural network, it is represented by N_w neurons, where neuron i has activity 1, and all others have activity 0
- We train the input to hidden matrix $\mathbf{V}$
- The input dimensionality is N_w
- For the purpose of the further discussion, we decompose

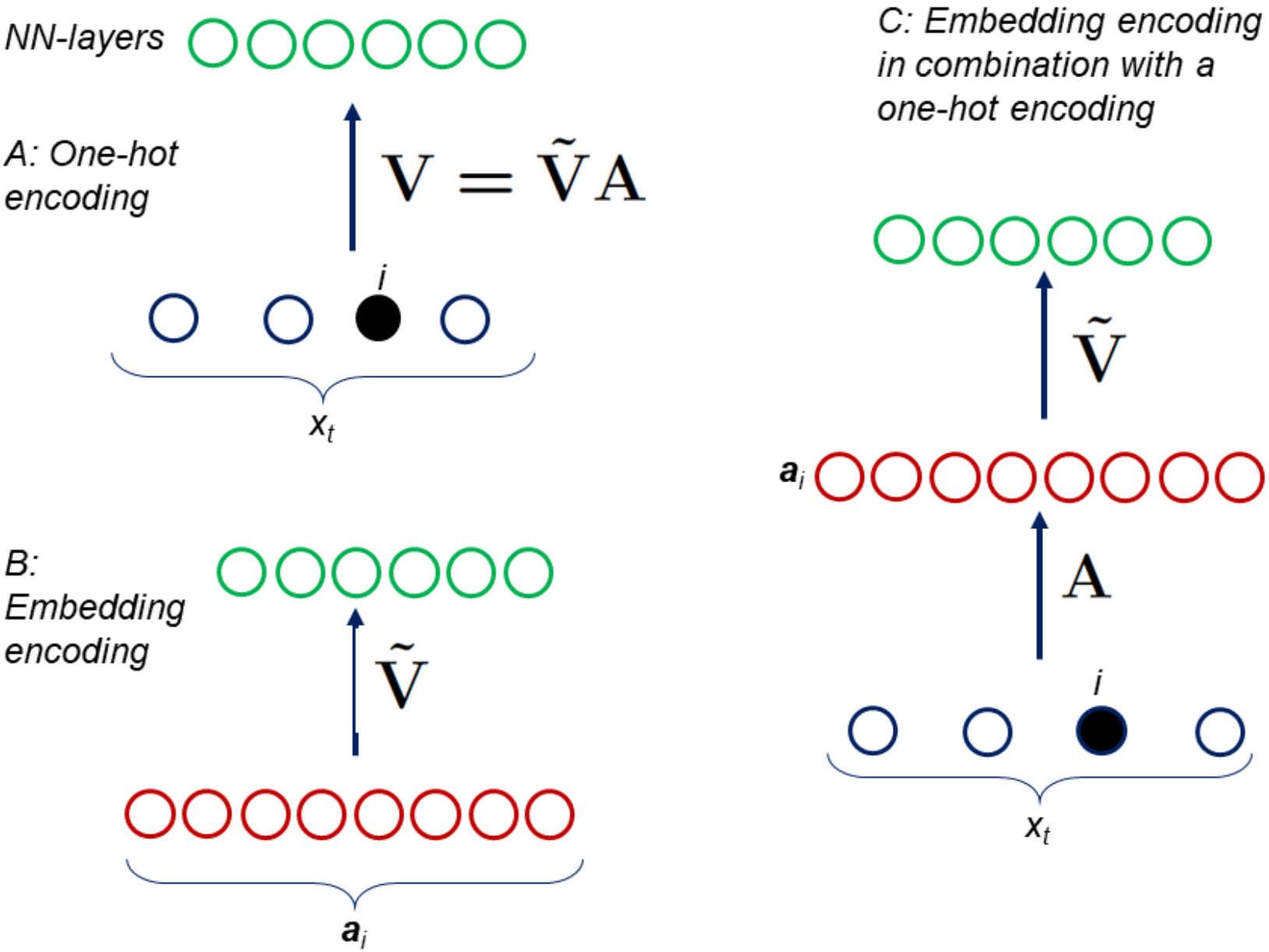
$$\mathbf{V} = \tilde{\mathbf{V}}\mathbf{A}$$

B: Embedding Encoding

- We know the embedding vector $\mathbf{a}_i$ for word i;
- This embedding vector might have been generated by some other research group and is simply a vector of real numbers of length r
- We can now use the embedding vector as input; think of the embedding vector as an attribute vector describing word i
- Now the input to hidden connection matrix is $\tilde{\mathbf{V}}$
- The input dimensionality is r

C: Embedding Encoding in Combination with One-hot Encoding

- The i-th column in matrix $\mathbf{A}$ contains $\mathbf{a}_i$
- If $\tilde{\mathbf{V}}$ is the weight matrix from representation layer to the neural hidden layer, then the weight matrix from one-hot encoding to hidden is $\mathbf{V} = \tilde{\mathbf{V}}\mathbf{A}$
- The input dimensionality is N_w
- The mapping from one-hot encoding to embedding layer can reduce the dimensionality considerably, when $r << N_w$ (bottleneck!)


NN-layers
A: One-hot encoding
$\mathbf{V} = \tilde{\mathbf{V}}\mathbf{A}$
i
x_t
B: Embedding encoding
$\tilde{\mathbf{V}}$
$\boldsymbol{a}_i$
C: Embedding encoding in combination with a one-hot encoding
$\tilde{\mathbf{V}}$
$\boldsymbol{a}_i$
$\mathbf{A}$
i
x_t

IIa. Representation Models and Language Models

Language Model

- The ideas is to predict the next word (out off a vocabulary of N_w words) in a text, based on the last T words
- Consider we want to predict y_t: y_t has as N_w components, one for each word (one-hot encoding)
- The inputs to the models are past words; the model assumption is that a word i is associated with an embedding vector $\mathbf{a}_i$ of dimension r (embedding representation)
- Thus in a first step, a one-hot encoding word i is mapped to the embedding vector of word $\mathbf{a}_i$ which is then the input to a neural network (embedding with one-hot)

Language Model (cont'd)

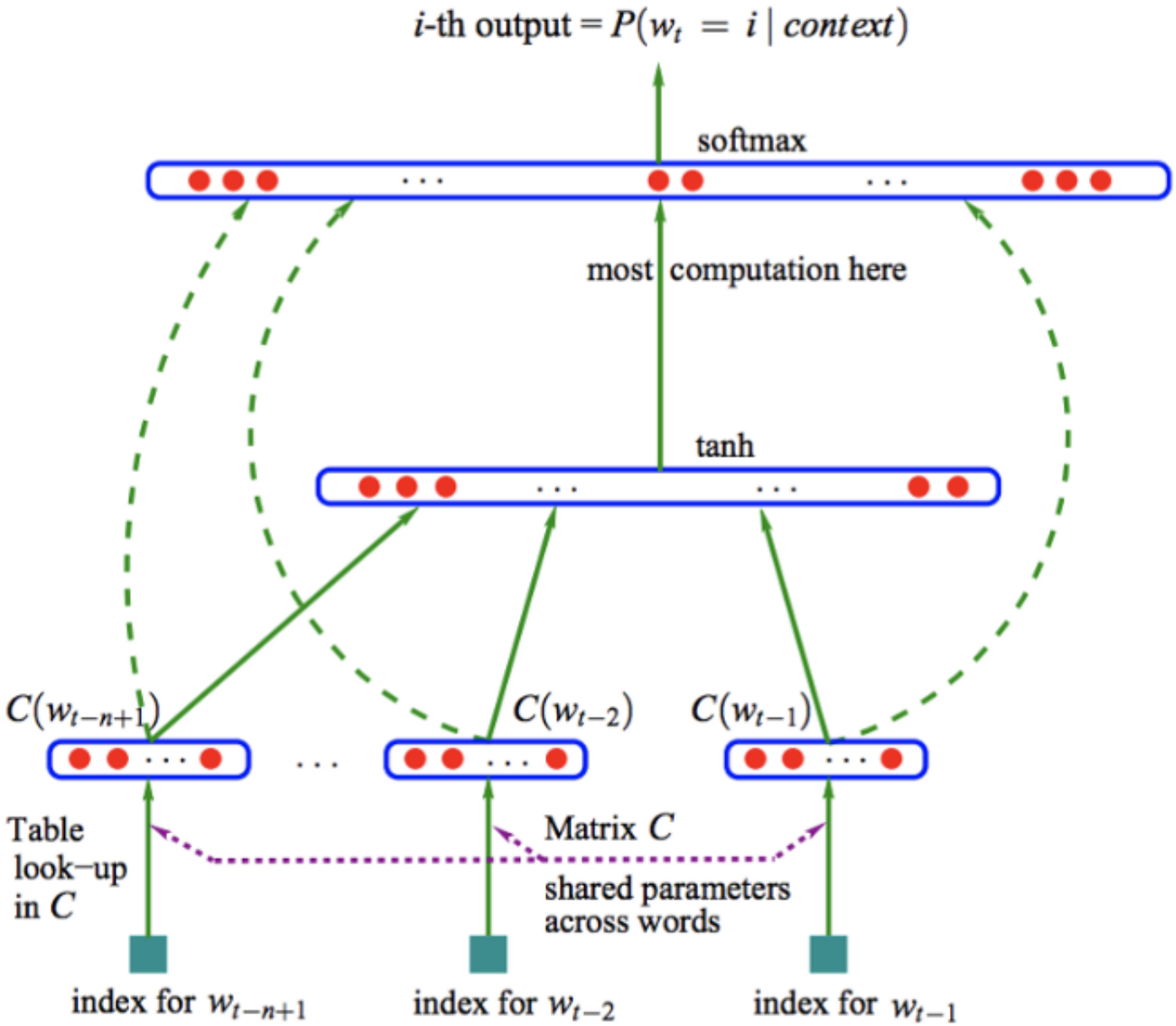
- We get

$$P(y_t = k | y_{t-1}, \ldots, y_{t-T}) = \textit{softmax}_k \left(\mathbf{f_w}(\mathbf{a}_{i(t-1)}, \ldots, \mathbf{a}_{i(t-T)}) \right)$$

where $i(t-m)$ is the index of the word at position $t-m$ and where $\mathbf{f_w}(\cdot)$ is a neural network with one hidden layer and N_w output neurons

Embeddings

- Training of the *word embeddings* and *the neural network parameters* can be done self-supervised on a huge corpus (without human labelling)
- After training, one obtains latent word representations (word embeddings) which are published and can be used in other applications
- State of the art are embeddings derived from language models like: ELMo, BERT, Word2vec, and GloVe
- The embedding idea is extremely powerful and one of the corner stones of modern machine learning
- In the next figure, the word embedding matrix is denoted as C


i-th output = $P(w_t = i \mid context)$
softmax
most computation here
tanh
$C(w_{t-n+1})$
$C(w_{t-2})$
$C(w_{t-1})$
Table look−up in C
Matrix C
shared parameters across words
index for w_{t-n+1}
index for w_{t-2}
index for w_{t-1}

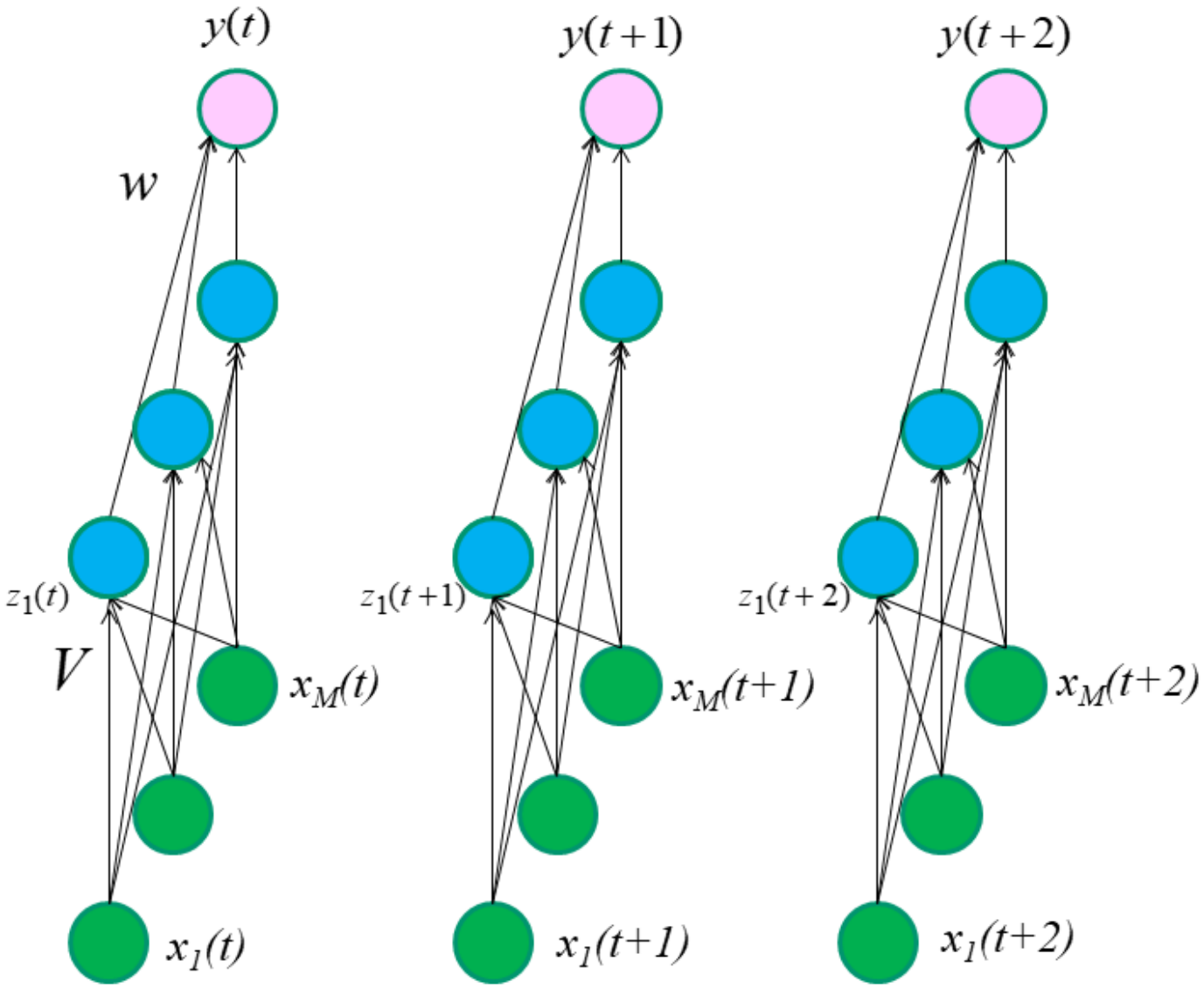
IIb. Recurrent Neural Networks

Recurrent Neural Network

- Recurrent neural networks (RNNs) are powerful methods for sequence modelling
- In their simplest form they are used to improve an output prediction by providing a memory for previous inputs

A Feedforward Neural Network with a Time Index

- We start with a normal feedforward neural network where the pattern is a sequential index t



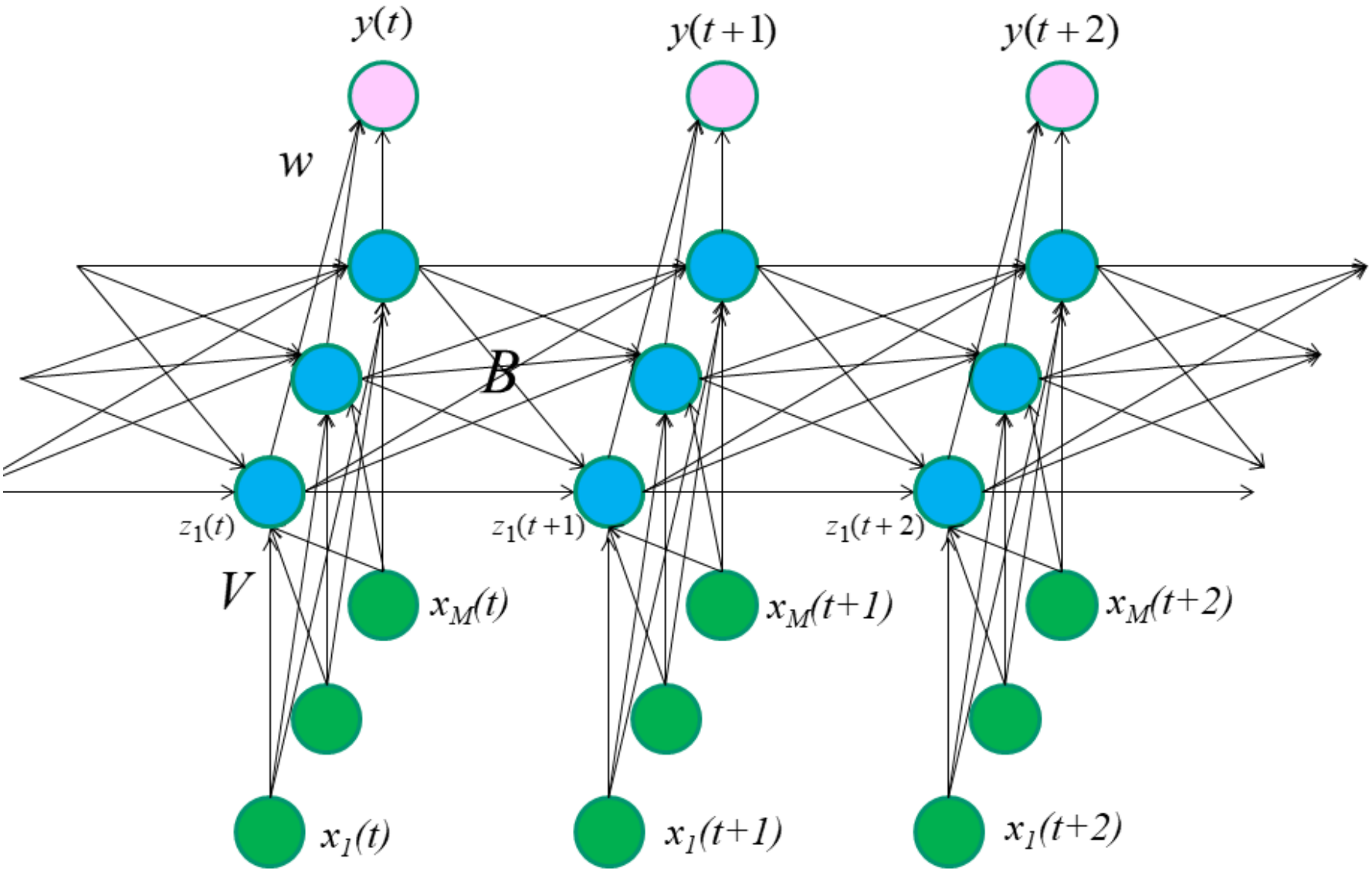
feed forward neural network at time step t

feed forward neural network at time step t+1

feed forward neural network at time step t+2

A Recurrent Neural Network Architecture unfolded in Time

- The hidden layer now also receives input from the hidden layer of the previous time step
- The hidden layer now has a memory function reflecting hidden inputs
- Thus a Recurrent Neural Network (RNN) is a nonlinear state-space model



Recurrent neural network, unfolded in time

A Recurrent Neural Network Architecture unfolded in Time (cont'd)

- In a compact notation, we write,

$$\mathbf{z}_t = \mathsf{sig}(B\mathbf{z}_{t-1} + V\mathbf{x}_t)$$

$$\mathbf{y}_t = \mathsf{sig}(W\mathbf{z}_t)$$

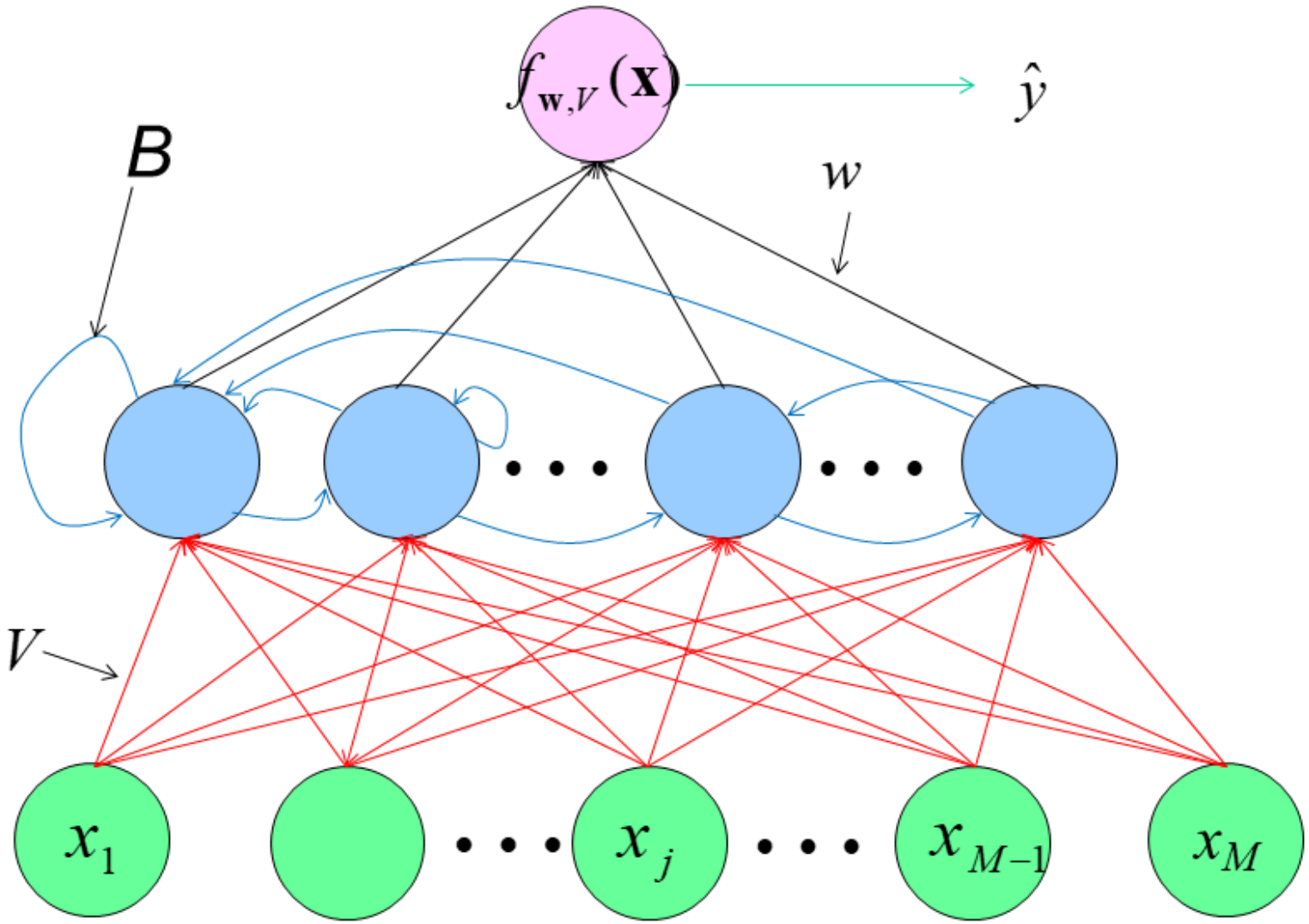
where we permit several outputs; also, in the last layer we might replace the sig with the *softmax*

Temporal Representation

- $\mathbf{z}_t$ is the representation of the state of the system (e.g., patient, plant, ...) at time t
- $\mathbf{x}_t$ can be the embedding of a thing which is present or active at time t (e.g., word, medication, ...)
- With several things, $\mathbf{x}_t$ can be generated from the embeddings of several things (all words in a sentence) by concatenation, averaging, ...
- This is a link to representation learning

Recurrent Representation

- The next slide shows an RNN as a recurrent structure



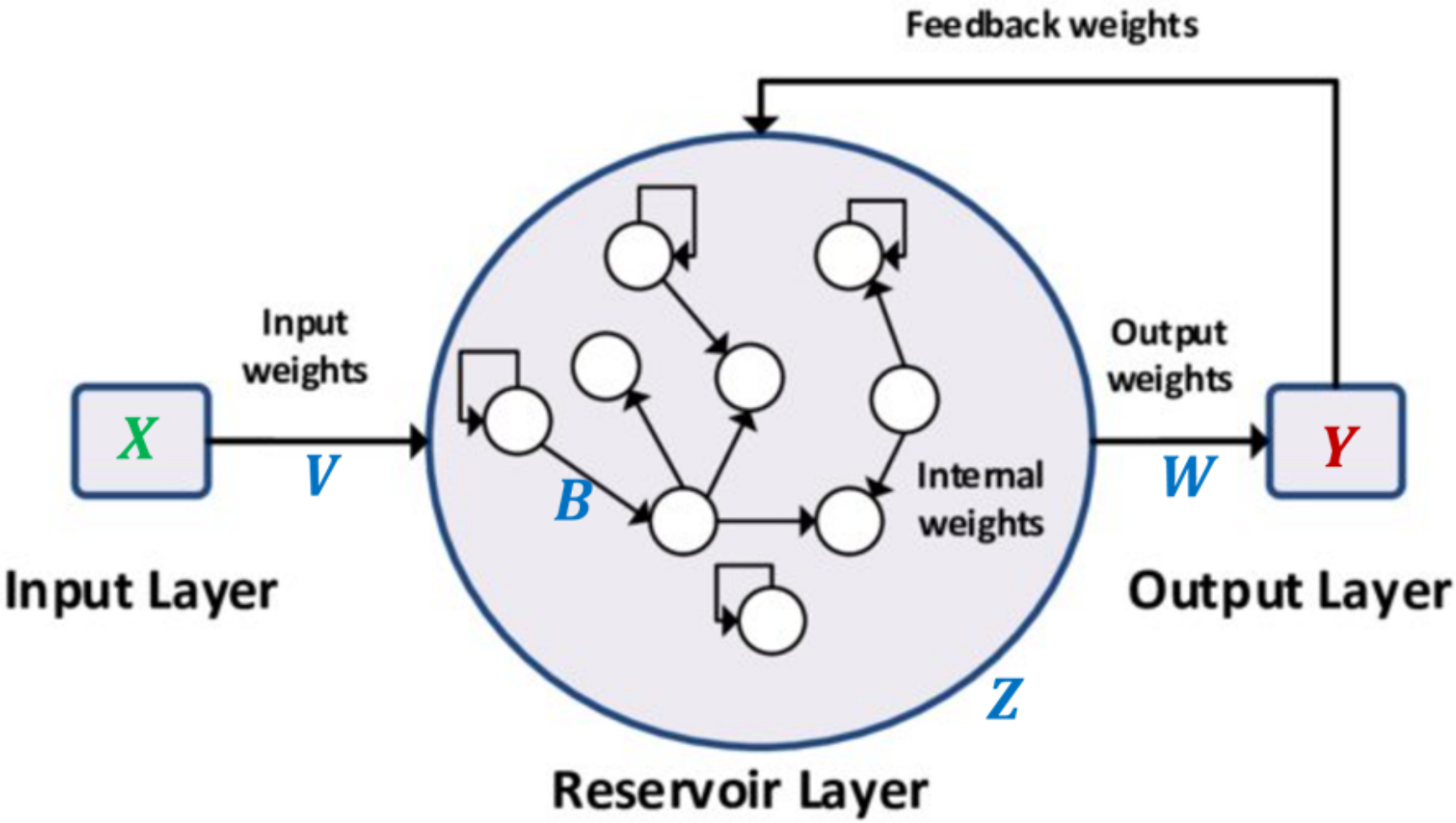
recurrent neural network showing recurrent connections

Backpropagation through time (BPTT)

- Training can be performed using backpropagation through time (BPTT), which is an application of backpropagation (SGD) to the unfolded network structure
- As an additional complexity, the error which occurs to the outputs at time t is not only backpropagated to the previous layers at time t, but also backward in time to all previous neural networks
- In principle, one would propagate back to $t = 1$; in practice, one typically truncates the gradient calculation

Echo-State Network

- Recurrent Neural Networks are sometimes difficult to train
- A simple alternative is to initialize B and V randomly (according to some recipe) and only train W
- W can be trained with the simple learning rules for linear regression or classification
- This works surprisingly well and is done in the Echo-State Network (ESN)
- ESN (and also liquid-state machines) are examples of so called *reservoir computing*


Feedback weights
Input weights
Output weights
X
V
B
Internal weights
W
Y
Input Layer
Output Layer
Z
Reservoir Layer

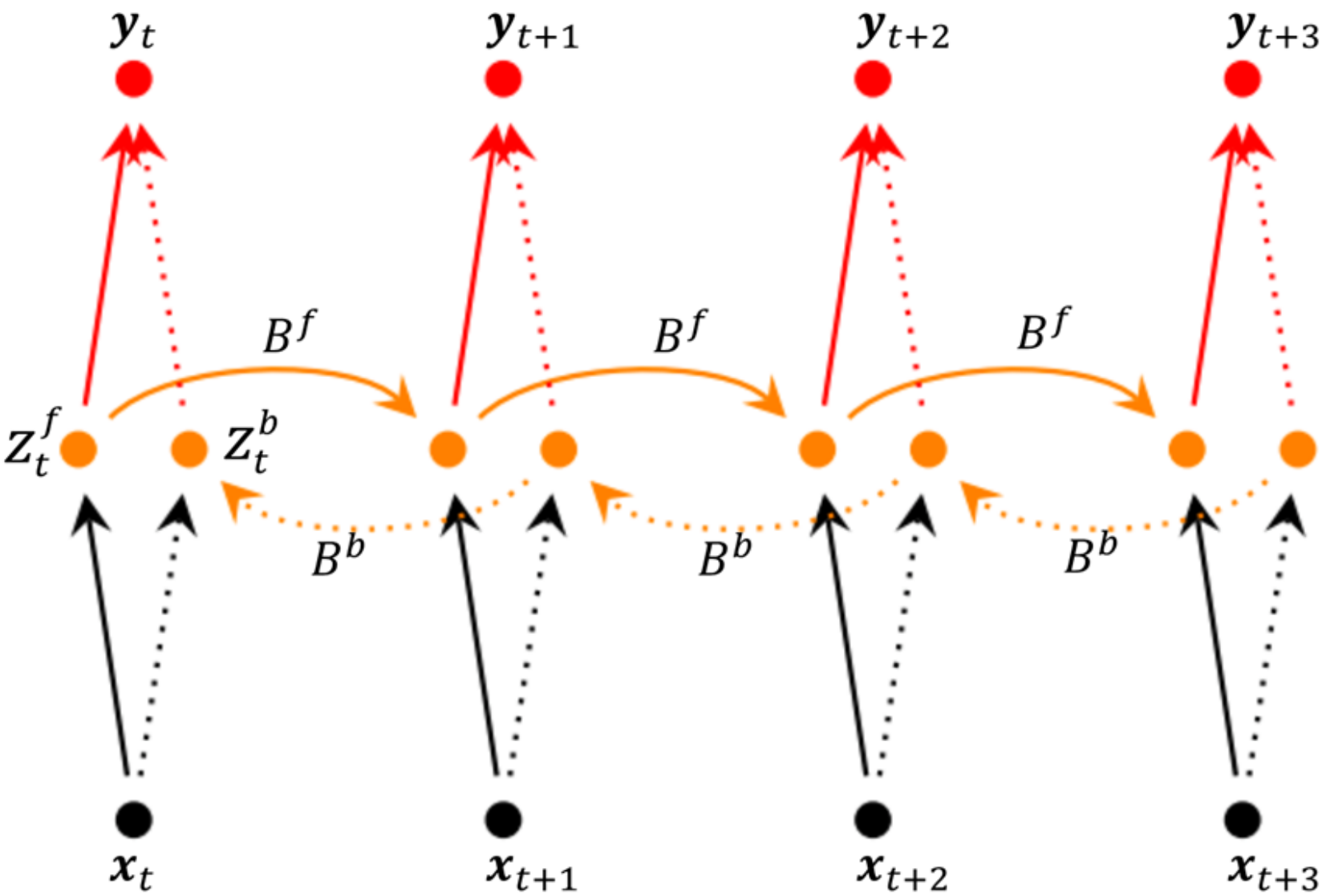
Issues in Prediction

- An RNN is typically used as predictive model in an iterative setting
- Due to the deterministic nature of the model: if the output $\mathbf{y}_t$ is predicted and then becomes available, it will not affect future predictions, since there is no information flowing back from $\mathbf{y}_t$ to $\mathbf{z}_t$
- This is in contrast to some probabilistic models such as hidden Markov models (HMMs), Kalman filters, stochastic state space models

Bidirectional RNNs

- The predictions in bidirectional RNNs depend on past and future inputs
- Useful for sequence labelling problems: handwriting recognition, speech recognition, bioinformatics, ...
- Bidirectional recurrent

$$\mathbf{z}_t = [\mathbf{z}_t^f; \mathbf{z}_t^b] = \left[\text{sig}\left(V^f \mathbf{x}_t + B^f \mathbf{z}_{t-1}^f\right); \text{sig}\left(V^b \mathbf{x}_t + B^b \mathbf{z}_{t+1}^b\right)\right]$$


$\boldsymbol{y}_t$
$\boldsymbol{y}_{t+1}$
$\boldsymbol{y}_{t+2}$
$\boldsymbol{y}_{t+3}$
B^f
B^f
B^f
Z_t^f
Z_t^b
B^b
B^b
B^b
$\boldsymbol{x}_t$
$\boldsymbol{x}_{t+1}$
$\boldsymbol{x}_{t+2}$
$\boldsymbol{x}_{t+3}$

IIc. LSTMS

Issues in Prediction

- Although the RNN has a memory, it has difficulties remembering important information far in the past
- This can be attributed to the vanishing gradient problem
- Solutions are the long short-term memory (LSTM), and the gated recurrent units (GRUs)
- We now discuss the LSTM

We Start with a Feedforward Neural Network

- Consider a feedforward neural network

$$\mathbf{s}_t = \text{sig}(V\mathbf{x}_t) \quad \mathbf{z}_t = \tanh(\mathbf{s}_t)$$

$$\hat{\mathbf{y}}_t = \text{sig}(W\mathbf{z}_t)$$

- The transfer function of the hidden neuron is a bit strange, $\tanh(\text{sig}(V\mathbf{x}_t))$
- $\mathbf{s}_t$ is called the **cell state vector**, $\mathbf{z}_t$ is the **output vector** (of the units, not the neural network)
- In the following steps, each latent unit will become an LSTM unit; thus we will have H LSTM units in the network

We Enter Input and Output Gates

- We now use input and output gates which can turn on and off individual LSTM units
- With input gate vector $\mathbf{g}_t$ and output gate vector $\mathbf{q}_t$

$$\mathbf{s}_t = \mathbf{g}_t \circ \mathsf{sig}(V\mathbf{x}_t) \qquad \mathbf{z}_t = \mathbf{q}_t \circ \mathsf{tanh}(\mathbf{s}_t)$$

  Here, $\circ$ is the elementwise (Hadamard) product. As before,

$$\hat{\mathbf{y}}_t = \mathsf{sig}(W\mathbf{z}_t)$$

- Input gates and output gates are also functions of the inputs

$$\mathbf{g}_t = \mathsf{sig}(V^g\mathbf{x}_t) \qquad \mathbf{q}_t = \mathsf{sig}(V^q\mathbf{x}_t)$$

- Gates are commonly used in mixture of expert neural networks, if the function switches between modes of operations

With Feedback

- We add recurrent connections to the cell state vector and the gates

$$\mathbf{s}_t = \mathbf{g}_t \circ \mathsf{sig}(V\mathbf{x}_t + B\mathbf{z}_{t-1}) \qquad \mathbf{z}_t = \mathbf{q}_t \circ \mathsf{tanh}(\mathbf{s}_t)$$

- Input Gate

$$\mathbf{g}_t = \mathsf{sig}(V^g\mathbf{x}_t + B^g\mathbf{z}_{t-1})$$

- Output Gate

$$\mathbf{q}_t = \mathsf{sig}(V^q\mathbf{x}_t + B^q\mathbf{z}_{t-1})$$

Cell State Vector with Self-recurrency and Forget Gate

- We add self-recurrency to the cell state vector, including a forget gate

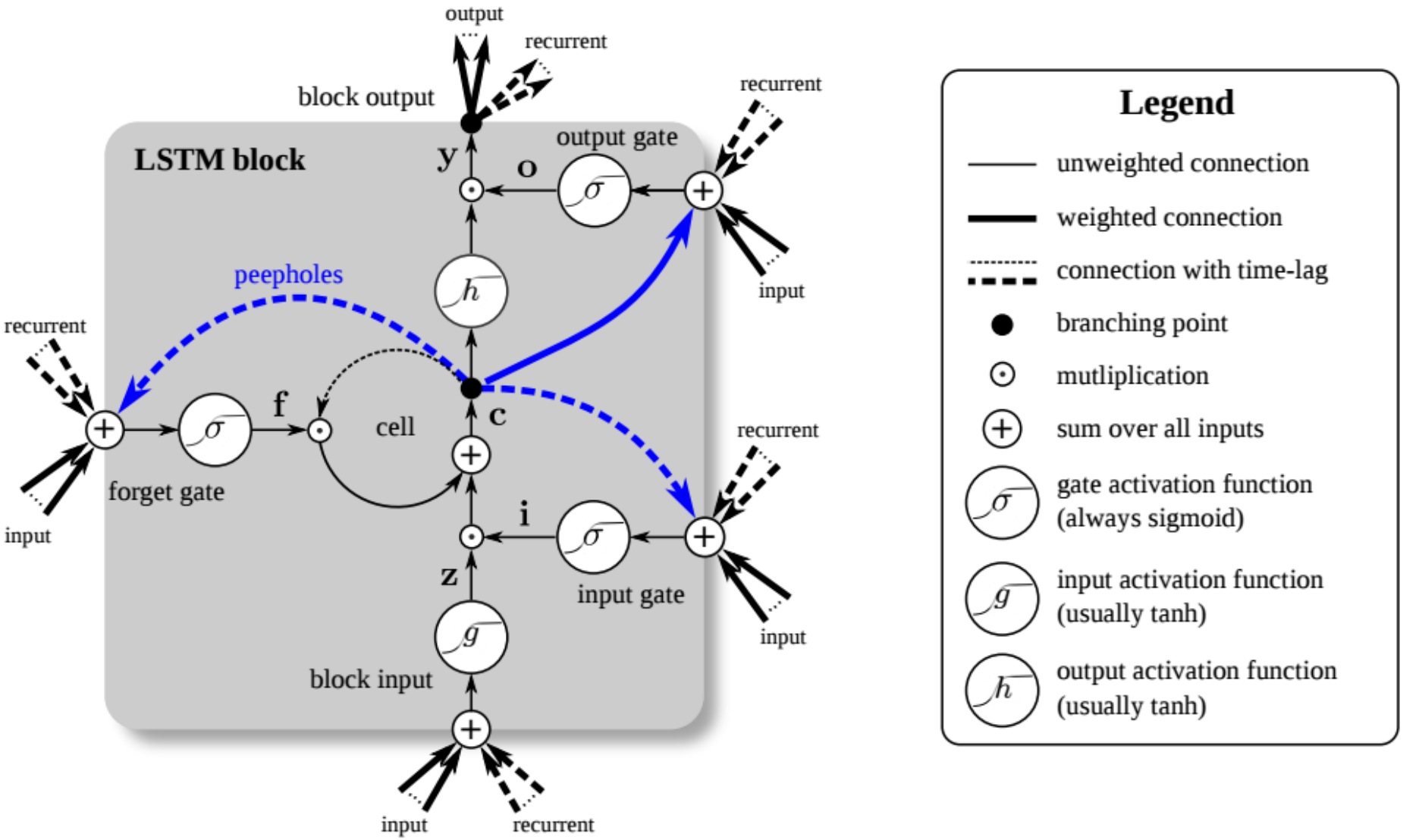
$$\mathbf{s}_t = \mathbf{f}_t \circ \mathbf{s}_{t-1} + \mathbf{g}_t \circ \mathsf{sig}(V\mathbf{x}_t + B\mathbf{z}_{t-1})$$

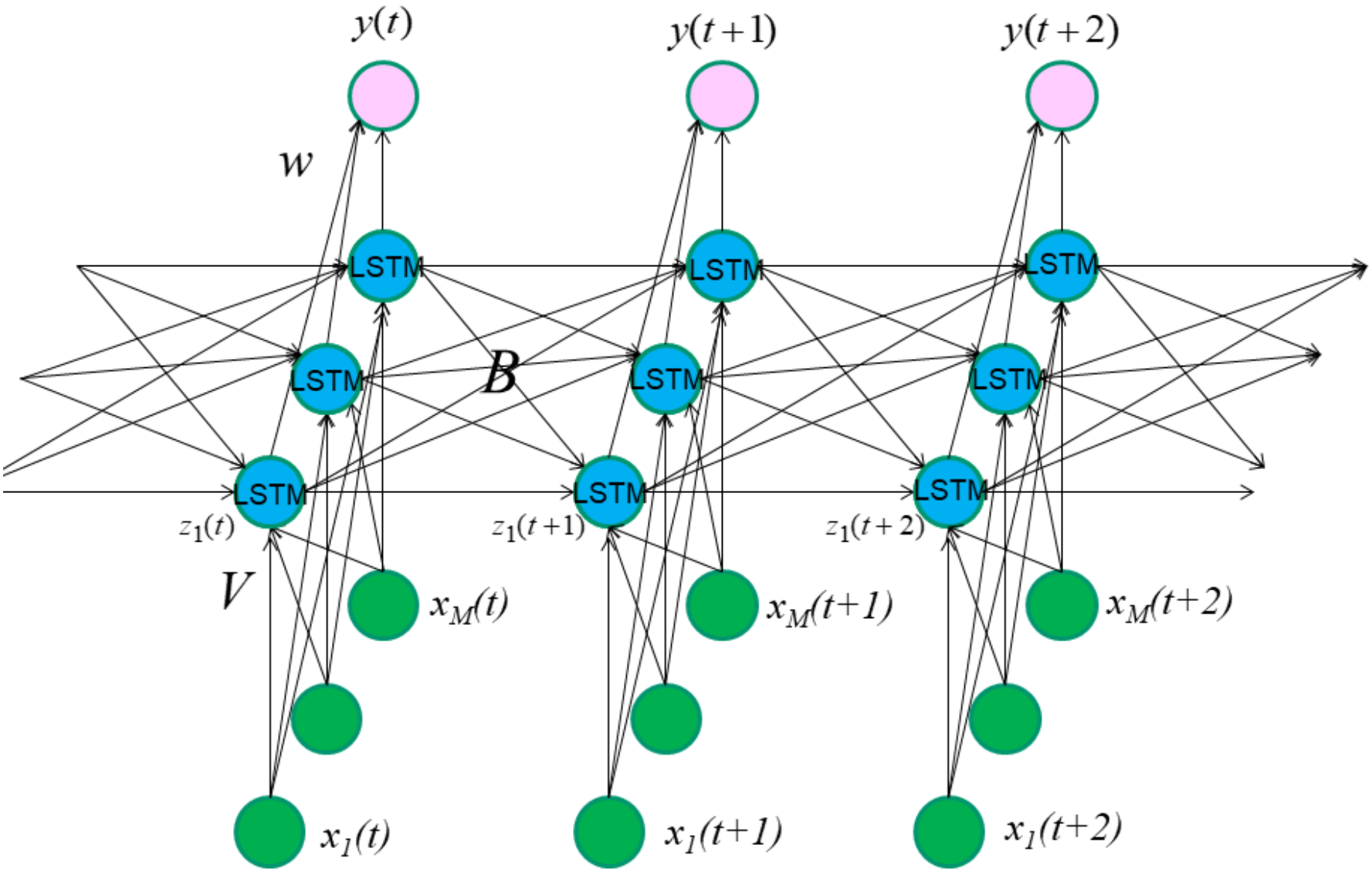
- Forget gate

$$\mathbf{f}_t = \mathsf{sig}(V^f\mathbf{x}_t + B^f\mathbf{z}_{t-1})$$

Long Short Term Memory (LSTM)

- As a recurrent structure the Long Short Term Memory (LSTM) approach has been very successful
- Basic idea: at time t a newspaper announces that the Siemens stock is labelled as "buy". This information will influence the development of the stock in the next days. A standard RNN will not remember this information for very long. One solution is to define an extra input to represent that fact and that is on as along as "buy" is valid. But this is handcrafted and does not exploit the flexibility of the RNN. A flexible construct which can hold the information is a long short term memory (LSTM) block.
- The LSTM was used very successful for reading handwritten text and is the basis for many applications involving sequential data (NLP, machine translation, ...)
- For the rest of the network, an LSTM node looks like a regular hidden node


output
recurrent
block output
recurrent
LSTM block
y
o
output gate
input
peepholes
recurrent
f
cell
c
recurrent
forget gate
input
i
z
input gate
input
block input
input
recurrent
Legend
unweighted connection
weighted connection
connection with time-lag
branching point
mutliplication
sum over all inputs
gate activation function
(always sigmoid)
input activation function
(usually tanh)
output activation function
(usually tanh)



Recurrent neural network, unfolded in time

LSTM Applications

- Wiki: LSTM achieved the best known results in unsegmented connected handwriting recognition, and in 2009 won the ICDAR handwriting competition. LSTM networks have also been used for automatic speech recognition, and were a major component of a network that in 2013 achieved a record 17.7% phoneme error rate on the classic TIMIT natural speech dataset

- Applications: Robot control, Time series prediction, Speech recognition, Rhythm learning, Music composition, Grammar learning, Handwriting recognition, Human action recognition, Protein Homology Detection

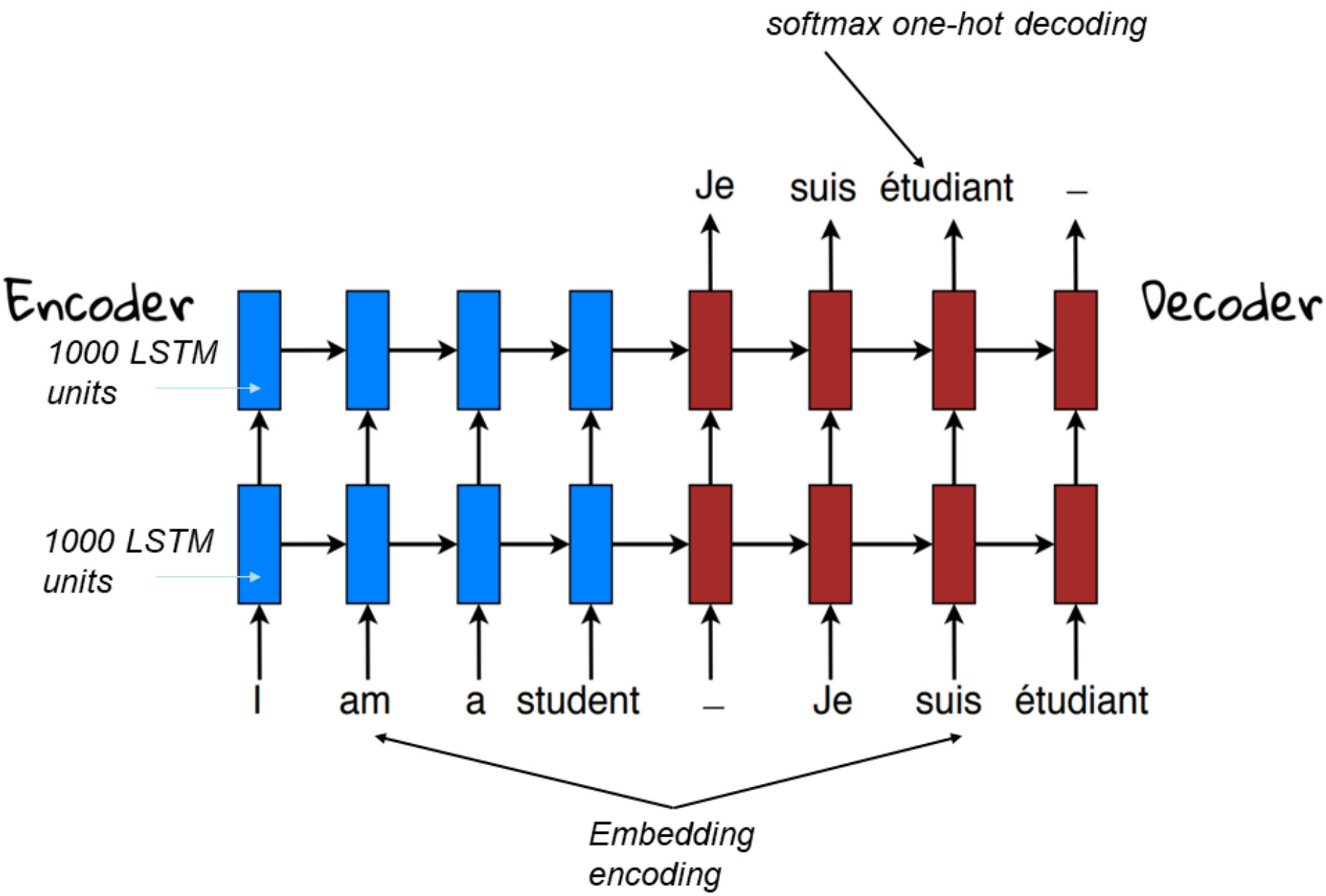
Comments on LSTM

- You cannot do transfer learning with LSTMs (does not work): thus you need a large data set for any new problem

IId. Encoder-Decoder Networks for Machine Translation

Encoder Decoder Architecture

- Most machine translation systems rely on the encoder-decoder approach
- Neural Machine Translation (NMT)
- Typical numbers: embedding rank: $r = 1000$, and 1000 hidden units per layer


softmax one-hot decoding
Je
suis
étudiant
–
Encoder
Decoder
1000 LSTM units
1000 LSTM units
I
am
a
student
–
Je
suis
étudiant
Embedding encoding

Encoder

- An **encoder** is an RNN (often an LSTM) with no output layer (no $\mathbf{y}_t$), but maybe several layers of recurrent units; as in the language model, *the inputs are latent embeddings of the words*
- The **encoder vectors** are the last hidden states (end-of-sentence)

Decoder

- The initial latent state of the decoder are the encoder vectors
- In its simplest form, the latent state of the decoder evolves as

$$\mathbf{z}_t = \text{sig}(B\mathbf{z}_{t-1} + V\mathbf{a}_{\mathbf{y}_{t-1}})$$

$$\mathbf{y}_t = \text{sig}(W\mathbf{z}_t)$$

- In training the input to the decoder is the *embedding of the previous word*; the output is the one-hot encoding of the current word
- Training is based on bilingual, parallel corpora; each hidden layer might consist of 1000 hidden units
- In testing one finds the most likely decoded sequence of words (e.g., using beam search); teacher forcing: the detected word appears at the input at the next instance
- Often one uses two or more hidden layers of LSTM units

Encoder-Decoder Approach in NMT

- Neural Machine Translation (NMT) achieved state-of-the-art performances in large-scale translation tasks such as from English to French
- NMT has the ability to generalize well to very long word sequences.
- The model does not have to explicitly store gigantic phrase tables and language models as in the case of standard MT; hence, NMT has a small memory footprint
- Implementing NMT decoders is easy unlike the highly intricate decoders in standard MT

IIe. Attention

Introduction

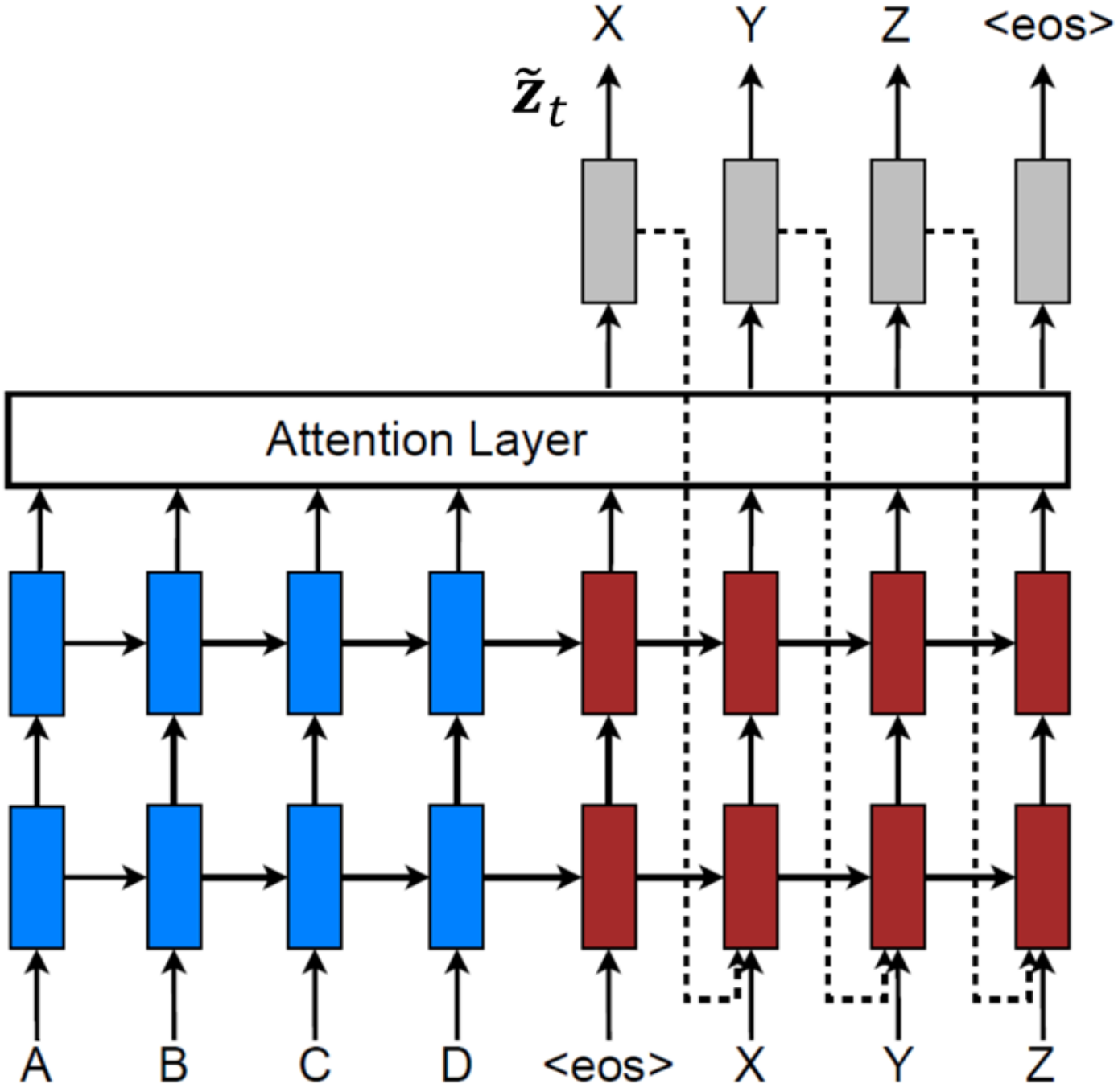
- The concept of "attention" has gained popularity recently in training neural networks, allowing models to learn alignments between different modalities, e.g., between image objects and agent actions in the dynamic control problem, between speech frames and text in the speech recognition task, or between visual features of a picture and its text description in the image caption generation task
- Attention has successfully been applied to jointly translate and align words
- Attention-based NMT models are superior to non attentional ones in many cases, for example in translating names and handling long sentences
- We follow: Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. 2016. "Effective Approaches to Attention-based Neural Machine Translation"
- First work: D. Bahdanau, K. Cho, and Y. Bengio. 2015. "Neural machine translation by jointly learning to align and translate." In ICLR

Bottleneck in the Encoder-Decoder Architecture

- In the encoder-decoder architecture, all information about the input sequence needs to be transported through the two embedding vectors
- Information earlier in the sequence tends to get forgotten
- One needs short cuts: maybe earlier embedding vectors are important as well!

Overall Architecture

- The next figure shows the overall architecture
- The attention layer sits on top of the normal encoder-decoder network
- Based on the neural activations in the encoder-decoder, it calculates new activations (grey boxes) in the fourth layer


X
Y
Z
<eos>
$\tilde{\mathbf{z}}_t$
Attention Layer
A
B
C
D
<eos>
X
Y
Z

Attention

- Let $\mathbf{z}_t$ (red) be a hidden state vector of interest in the **decoder** (so called target at t; also called the **query**)
- Let $\mathbf{c}_t$ be the source-side **context vector** (derived further down)
- The attentional hidden state (grey) is

$$\tilde{\mathbf{z}}_t = \text{sig}\left(V\mathbf{z}_t + D\mathbf{c}_t\right)$$

- The sig is typically the tanh; note that this is a normal layer in a neural network where the layer $\mathbf{z}_t$ is the lower layer and $\tilde{\mathbf{z}}_t$ is the upper layer and where the lower layer is appended with $\mathbf{c}_t$
- $\tilde{\mathbf{z}}_t$ is then the top hidden layer: the decoded word probability at the target is calculated as *softmax*$(W_s\tilde{\mathbf{z}}_t)$

Global Attention: What is the Context $\mathbf{c}_t$?

- Let $\bar{\mathbf{z}}_s$ be any activation vector in the encoder (source hidden state) (often restricted to the top layer) (the **key**)
- The **alignment** of s for t is a scalar,

$$a_t(s) = \mathit{align}(\mathbf{z}_t, \bar{\mathbf{z}}_s) = \frac{\exp(\mathit{score}(\mathbf{z}_t, \bar{\mathbf{z}}_s))}{\sum_{s'} \exp(\mathit{score}(\mathbf{z}_t, \bar{\mathbf{z}}_{s'}))}$$

- The alignment score function calculates a similarity measure: A typical **score** is the dot product, $score(\mathbf{z}_t, \bar{\mathbf{z}}_s) = \mathbf{z}_t^T \bar{\mathbf{z}}_s$
- The already introduced **context vector** is then calculated as (here, $\bar{\mathbf{z}}_s$ assumes the role of the **value**)

$$\mathbf{c}_t = \sum_s a_t(s) \bar{\mathbf{z}}_s$$

- The new parameters are contained in V and D; if in training it turns out that attention is useless, all entries in D converge to 0

Reformulation

- We can write

$$\tilde{\mathbf{z}}_t = \text{sig}\left(V\mathbf{z}_t + D\bar{Z}\text{softmax}\left(\bar{Z}^T\mathbf{z}_t\right)\right)$$

- The columns in $\bar{Z}$ are the sorce embedding vectors
- $\mathbf{z}_t$ might align (pay attention to) certain components of $\bar{Z}$

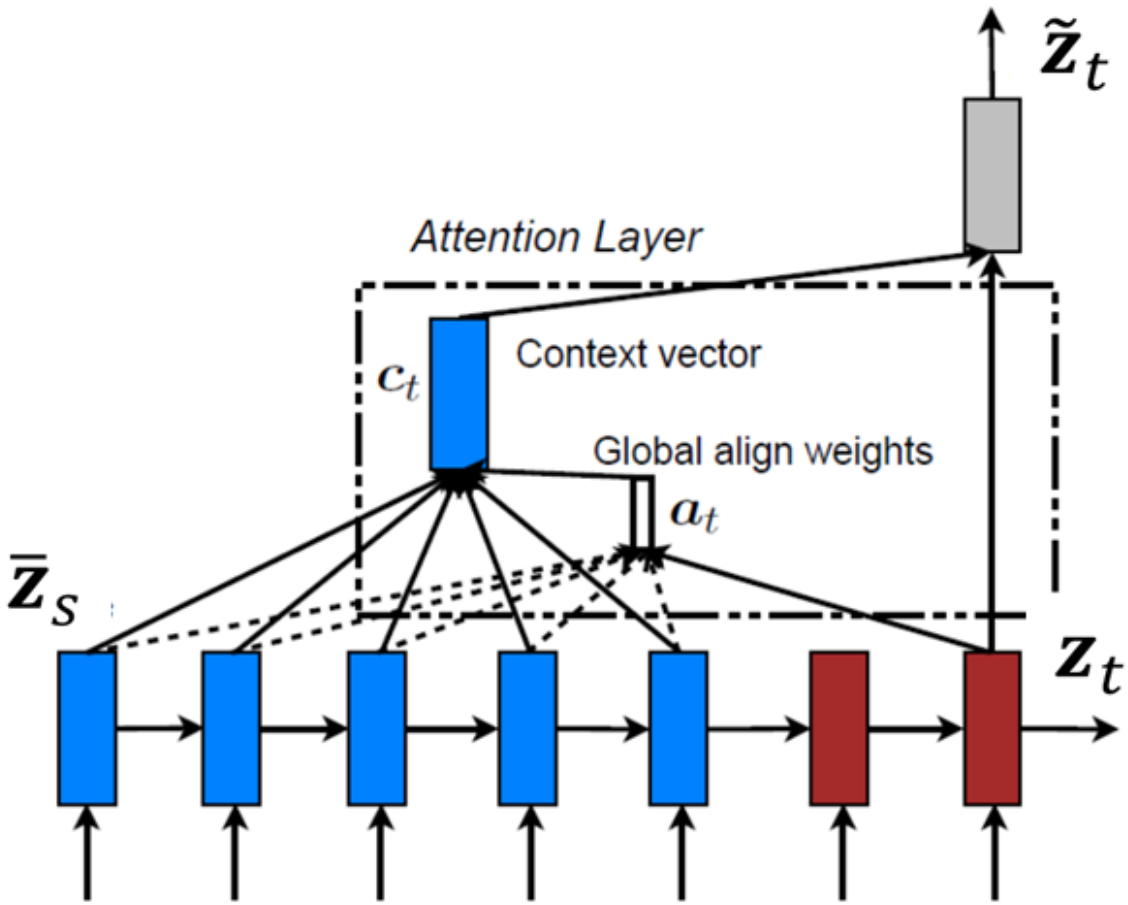


Figure 2: **Global attentional model** – at each time step t, the model infers a *variable-length* alignment weight vector $\boldsymbol{a}_t$ based on the current target state $\boldsymbol{z}_t$ and all source states $\bar{\boldsymbol{z}}_s$. A global context vector $\boldsymbol{c}_t$ is then computed as the weighted average, according to $\boldsymbol{a}_t$, over all the source states.

Local Attention Model (Position Encoding)

- The global attention has a drawback that it has to attend to all words on the source side for each target word, which is expensive and can potentially render it impractical to translate longer sequences, e.g., paragraphs or documents
- To address this deficiency, a local attentional mechanism has been proposed that chooses to focus only on a small subset of the source positions per target word
- The new alignment becomes

$$a_t(s) \propto \textit{align}(\mathbf{z}_t, \bar{\mathbf{z}}_s) \exp\left(\frac{(s - p_t)^2}{2\sigma^2}\right)$$

- p_t is the expected position in the input sequence predicted from $\mathbf{z}_t$ using a neural network

$$p_t = S\text{sig}(v_p^T \tanh(W_p \mathbf{z}_t))$$

S is the source sentence length

Local Position Encoding

- Alternatively: a vector is defined which encodes the position of a token
- This vector is added (summed) to the word encoding embedding

Sequence	Index of token, k	Positional Encoding Matrix with d=4, n=100			
		i=0	i=0	i=1	i=1
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

Position Encoding

- We have

$$a_{k,2i} = \sin((k-1)/n^{2i/T})$$

$$a_{k,2i+1} = \sin((k-1)/n^{2i/T})$$

- $i = 1, \ldots, T/2$. Often $n = 10000$. Thus we start with constants and the smallest wavelength is $2\pi n$

- I need the nonlinear dependency to squeeze 10000 frequencies into a vector of dimension r

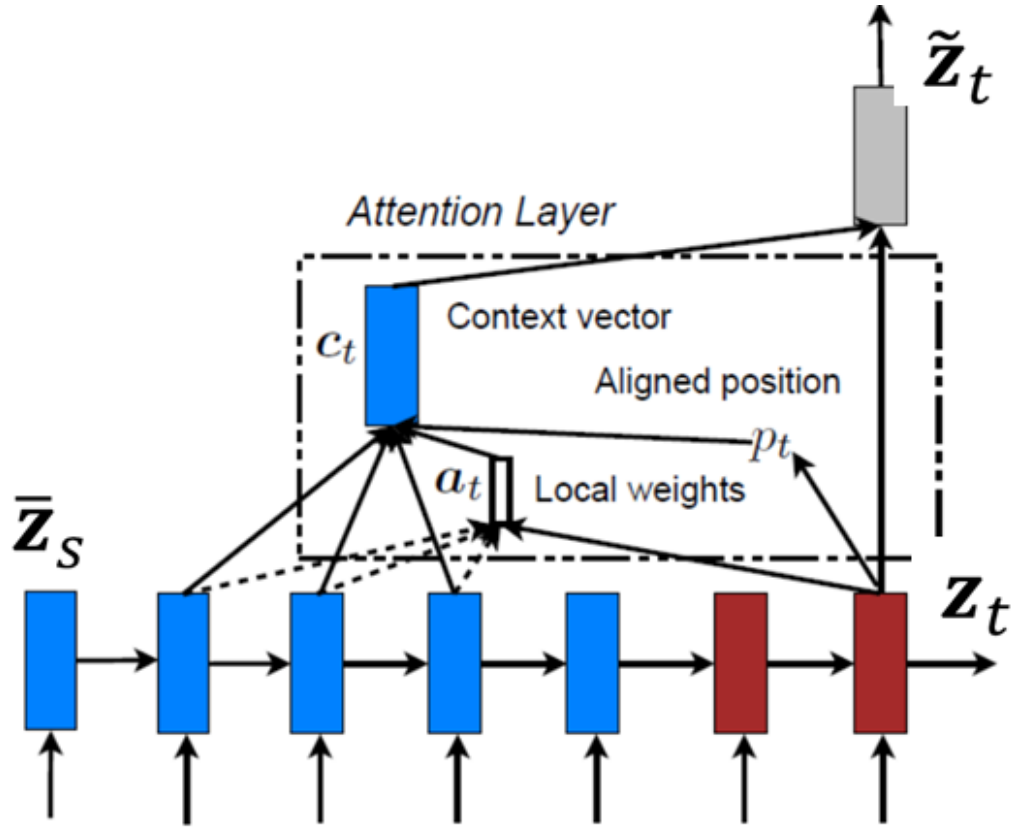
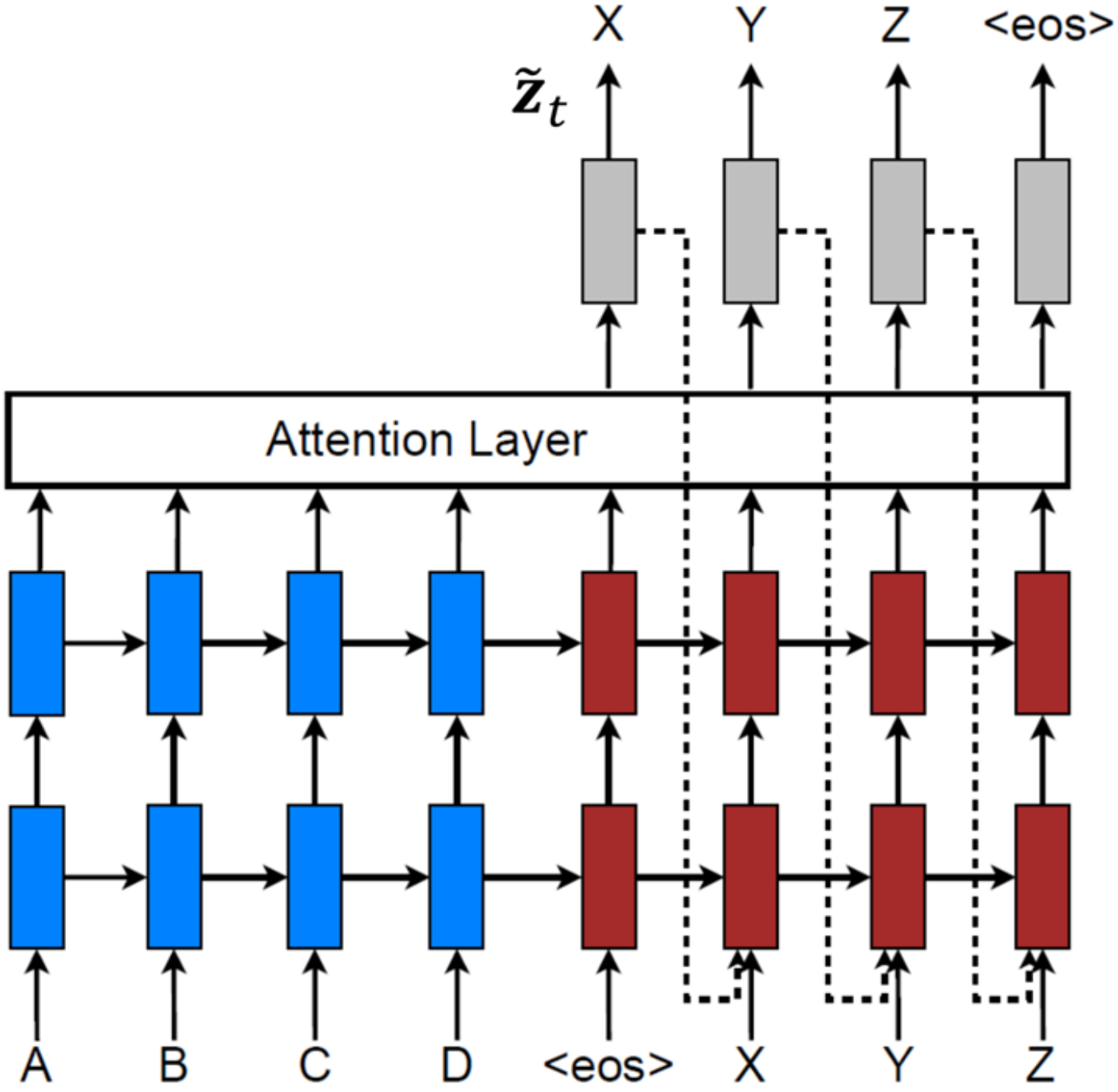


Figure 3: **Local attention model** – the model first predicts a single aligned position p_t for the current target word. A window centered around the source position p_t is then used to compute a context vector $\mathbf{c}_t$, a weighted average of the source hidden states in the window. The weights $\mathbf{a}_t$ are inferred from the current target state $\mathbf{z}_t$ and those source states $\bar{\mathbf{z}}_s$ in the window.

Overall Architecture

- The next figure shows again the architecture (ignore the dashed lines)


X
Y
Z
<eos>
$\tilde{\mathbf{z}}_t$
Attention Layer
A
B
C
D
<eos>
X
Y
Z

From Attention to Self-Attention

- So far we calculated the attention of an element in the output sequence w.r.t all elements in the input sequence
- Let's consider another task, e.g., entity labeling
- We have a sequence of words/entities ($t = 1, 2, ...$) as inputs; the goal is to provide a label for each word/entity, or to provide a label for the whole sequence
- We now change notation: $\mathbf{z}_{t,l}$ is the activation vector at layer l
- Self-attention can be applied to any deep neural network
- Self-attention can replace convolutional and recurrent approaches ("attention is all you need")

Self-Attention (cont'd)

- In self-attention, the activation of a hidden layer $\mathbf{z}_{t,l}$ is calculated based on other layer's $\mathbf{z}_{t,l-1}$ of all entities/data points as

$$\mathbf{z}_{t,l} = \text{sig}\left(V_l \mathbf{z}_{t,l-1} + D_l \mathbf{c}_{t,l-1}\right)$$

- Here, the context vector is

$$\mathbf{c}_{t,l-1} = \sum_{t'} a\Big(\mathbf{z}_{t,l-1}, \mathbf{z}_{t',l-1}\Big)\, \mathbf{z}_{t',l-1}$$

  The sum is over all elements in the sequence

- (Often the tanh is used instead of the sig)

Comparison

- **Feed forward neural network**

$$\mathbf{z}_{t,l} = \text{sig}\left(V_l \mathbf{z}_{t,l-1}\right)$$

so here each word label at position t is predicted separately; embeddings are all independent; this is the i.i.d situation

- **Fully connected** (not used in practice)

$$\mathbf{z}_{t,l} = \text{sig}\left(V_l \mathbf{z}_{t,l-1} + \sum_{t'} C_{t,t',l} \mathbf{z}_{t',l-1}\right)$$

The embeddings of all words are considered; here one would need to use a standard length sentence (short sentences are dealt with by zero-passing); a problem with this approach is the huge number of parameters in the neural network

Comparison (cont'd)

- **Convolutional layer**

$$\mathbf{z}_{t,l} = \text{sig}\left(V_l \mathbf{z}_{t,l-1} + \sum_k \sum_{t'} C^k_{t-t',l} \mathbf{z}_{t',l-1} \right)$$

  Very powerful approach and very successful in NLP; needs zero padding at sentence boundaries

- In some approaches (e.g., graph convolution) simply the averages of the neighbor embeddings are calculated

Comparison (cont'd)

- **Recurrent neural networks**

$$\mathbf{z}_{t,l} = \text{sig}\left(V_l \mathbf{z}_{t,l-1} + B_l \mathbf{z}_{t-1,l}\right)$$

Very powerful approach and very successful in NLP; often LSTM units are used

- **Bidirectional recurrent neural networks**

$$\mathbf{z}_{t,l} = [\mathbf{z}^f_{t,l}; \mathbf{z}^b_{t,l}]$$

$$= \left[\text{sig}\left(V_l^f \mathbf{z}_{t,l-1} + B_l^f \mathbf{z}^f_{t-1,l}\right); \text{sig}\left(V_l^b \mathbf{z}_{t,l-1} + B_l^b \mathbf{z}^b_{t+1,l}\right)\right]$$

Comparison (cont'd)

- **Self-Attention**

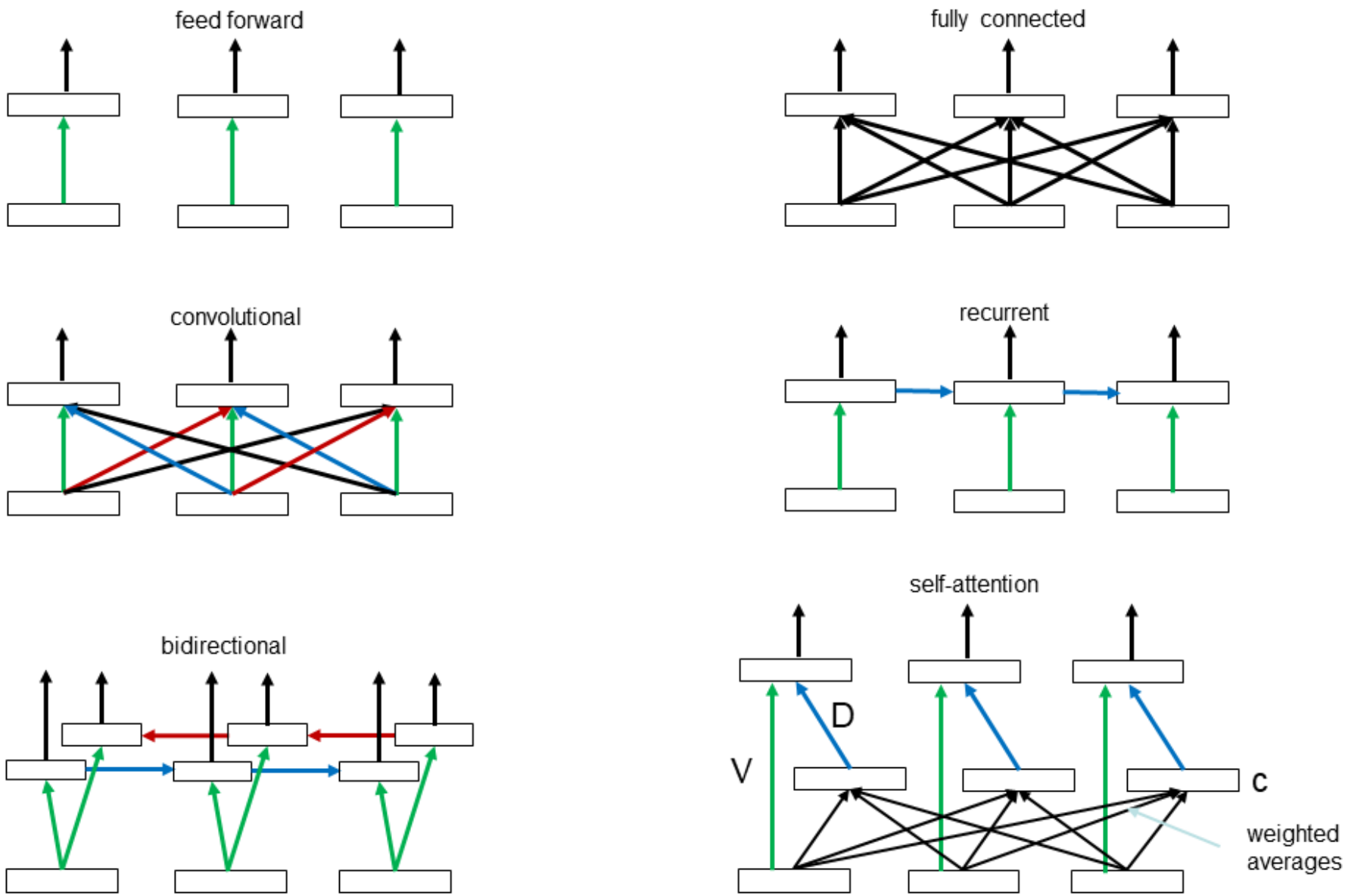
$$\mathbf{z}_{t,l} = \text{sig}\left(V_l \mathbf{z}_{t,l-1} + D_l \mathbf{c}_{t,l-1}\right)$$

$$\mathbf{c}_{t,l-1} = \sum_{t'} a\Big(\mathbf{z}_{t,l-1}, \mathbf{z}_{t',l-1}\Big)\ \mathbf{z}_{t',l-1}$$

(the sig is often the tanh) self-attention can replace convolutional or recurrent layers

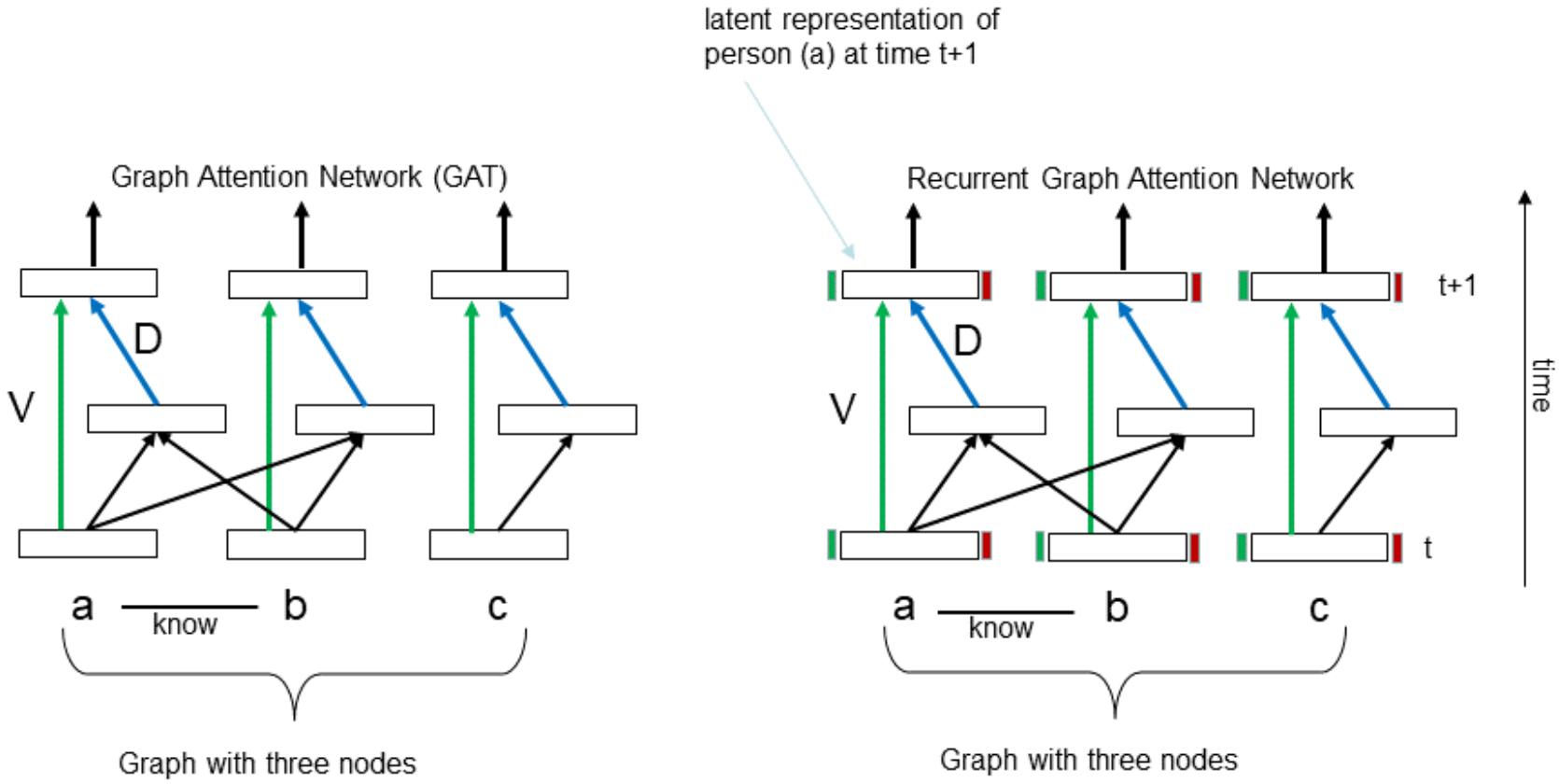
- Compare to graph convolution networks (GCNs): here $\mathbf{c}_{t,l-1}$ is the average over the neighbors in the graph, including $\mathbf{z}_{t,l}$; $V = 0$;

- GCNs with attentions: Graph Attention Networks: enhances GCNs with an attention mechanism


feed forward
fully connected
convolutional
recurrent
bidirectional
self-attention
D
V
c
weighted
averages

Towards Graph Neural Networks

- Consider that each neural network represents an entity, like a student at a university
- Then we have a network structure, i.e. two students are connected, when they know each other
- In this case, the attention mechanism can be restricted to neighbors in the graph
- This is a Graph Attention Network (GAT), an example of a Graph Neural Network (GNN)


latent representation of
person (a) at time t+1
Graph Attention Network (GAT)
Recurrent Graph Attention Network
D
V
t+1
t
time
a
know
b
c
Graph with three nodes
Graph with three nodes

Towards a Recurrent Graph Attention Network

- Same as before but a layer gets a time index and possibly a local input (green) and output (red)

Conclusions

- Sequential models find many applications in natural language processing (NLP) applications, including machine translation
- Attention mechanisms are the basis for state of the art machine translation (Transformer) and context sensitive embedding models; BERT (Bidirectional Encoder Representations from Transformers)

Transformer, BERT, GPT

Transformer

- Bottleneck of previous approaches in NMT: sequential processing at the encoding step
- The Transformer **dispensed the recurrence and convolutions** involved in the encoding step entirely and based models only on attention mechanisms to capture the global relations between input and output
- Each layer has two sub-layers comprising multi-head attention layer followed by a position-wise feed forward network.

Context with Learned Projection Matrices

- Consider self-attention with (all representations and all parameter matrices depend on layer l, which we do not explicitly indicate to simplify notation)

$$\mathbf{c}_t = \sum_{t'} a\Big(W^Q \mathbf{z}_t, W^K \mathbf{z}_{t'}\Big)\, W^V\, \mathbf{z}_{t'}$$

- The W^Q, W^K, W^V, are projection matrices for the **query** $\mathbf{z}_t$, the **key** $\mathbf{z}_{t'}$ and the **value** $\mathbf{z}_{t'}$

- Thus the calculation of the context vector (still a vector!) involves tunable matrices

Multi-head Attention: Several Context Vectors

- Now we define L ($k = 1, \ldots, L$) context vectors, also called heads

$$\textit{head}_{k,t} = \mathbf{c}_{k,t} = \sum_{t'} a\left(W_k^Q \mathbf{z}_t, W_k^K \mathbf{z}_{t'}\right) W_k^V \ \mathbf{z}_{t'}$$

- With L heads,

$$\mathbf{c}_t = W^O[\textit{head}_{1,t}; \cdots; \textit{head}_{L,t}]$$

  Thus there are L different attention mechanisms which are appended and multiplied by the $r \times (L \times r)$ matrix W^O, where r is the embedding dimension

- Note that $\mathbf{c}_t$ is again a single vector of dimension r

Encoder: Multi-head Self Attention (cont'd)

- Then it uses a simple feedforward neural network $\mathbf{f_w}(\cdot)$ to compute

$$\mathbf{z}_{t,l} = \ddot{\mathbf{z}}_{t,l-1} + \mathbf{f_w}\left(\ddot{\mathbf{z}}_{t,l-1}\right)$$

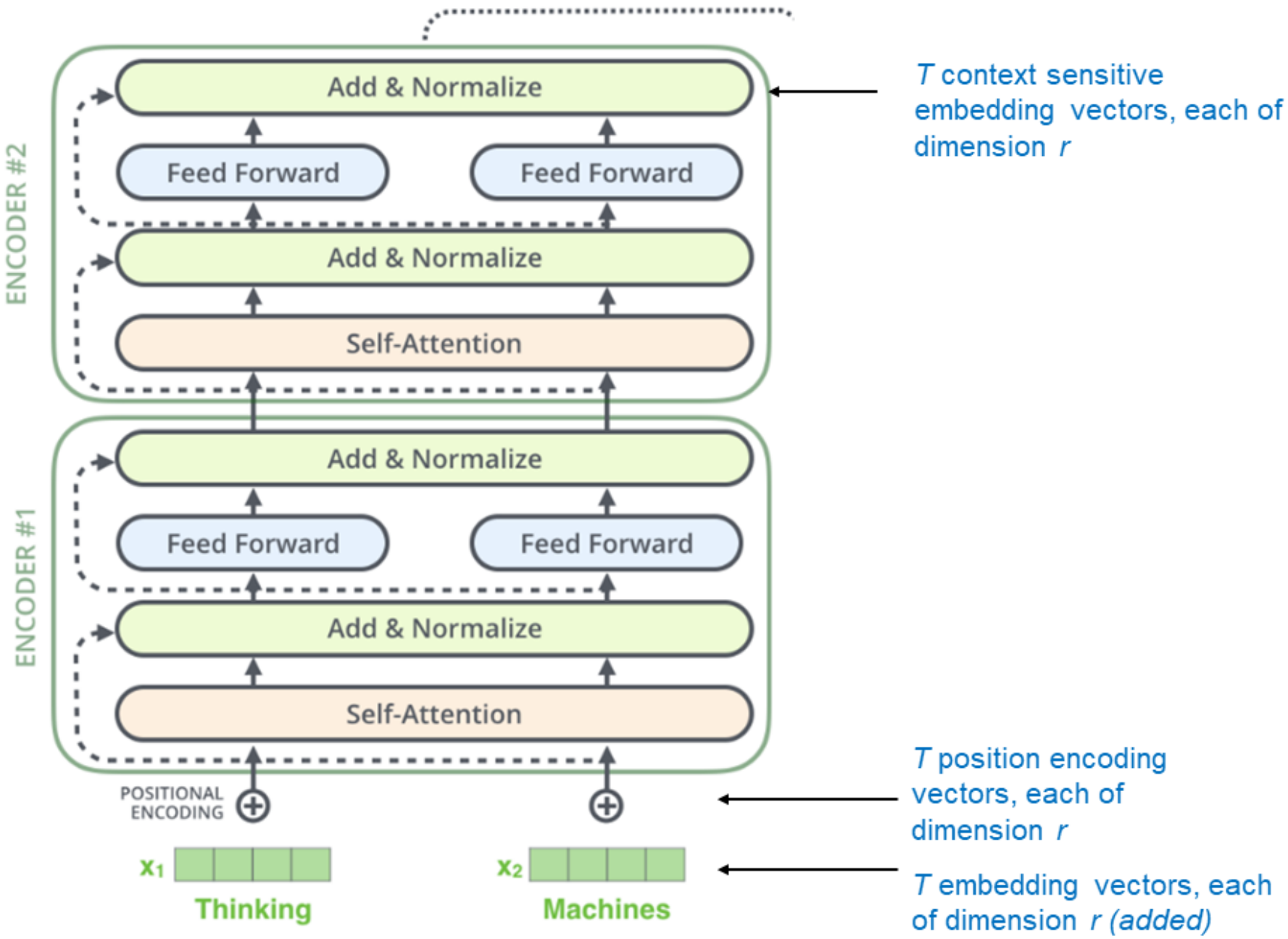
with

$$\ddot{\mathbf{z}}_{t,l-1} = V_l \mathbf{z}_{t,l-1} + D_l \mathbf{c}_{t,l-1}$$

- Thus it uses ResNet-type processing

Encoder (cont'd)

- Layer Normalization is used (normalizes the activations, like batch normalization)
- Positional Encoding: To address this, the transformer adds a vector to each input embedding. These vectors follow a specific pattern that the model learns, which helps it determine the position of each word, or the distance between different words in the sequence.
- Without positional encoding, the transformer (and BERT) would be a bag-of-words approach and could not distinguish between "Live to Work" and "Work to Live", which an LSTM could!


ENCODER #2
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Self-Attention
ENCODER #1
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Self-Attention
POSITIONAL ENCODING
x1
x2
Thinking
Machines
T context sensitive embedding vectors, each of dimension r
T position encoding vectors, each of dimension r
T embedding vectors, each of dimension r (added)

Sequence	Index of token, k	Positional Encoding Matrix with d=4, n=100			
		i=0	i=0	i=1	i=1
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

Decoder: Self Attention

- The inputs are the position encoding embeddings
- The decoder uses a **multi-head self-attention layer**, as in the encoder

Decoder: Encoder-decoder Attention Layer

- The decoder also has a **multi-head encoder-decoder attention layer**
- Attention is w.r.t the $\{\mathbf{z}_{t,out}\}_{t=1}^{T}$ from the encoder (thus only its output representation vectors!)

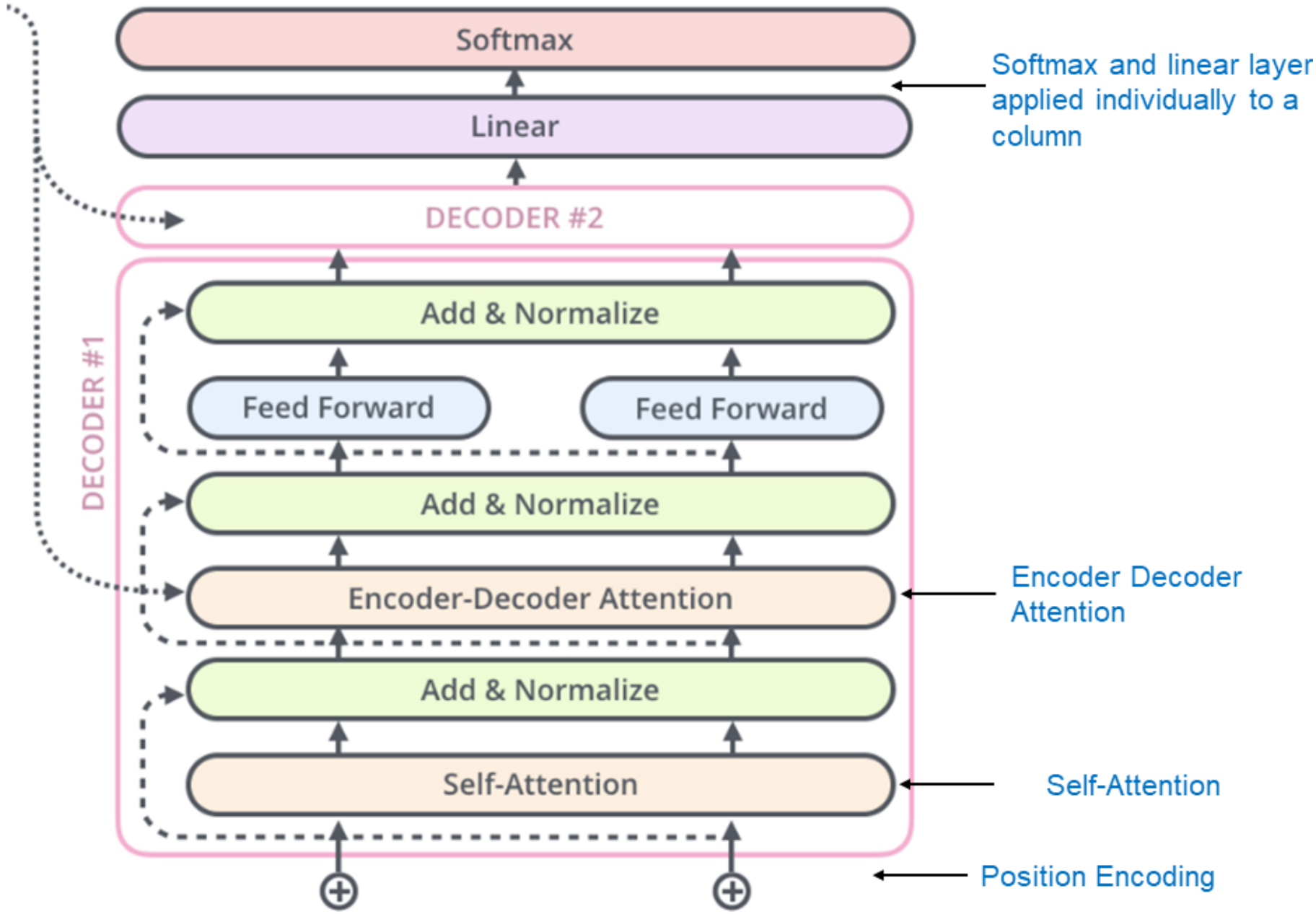
Multi-head Encoder-decoder Attention

- Multi-head encoder-decoder attention uses:

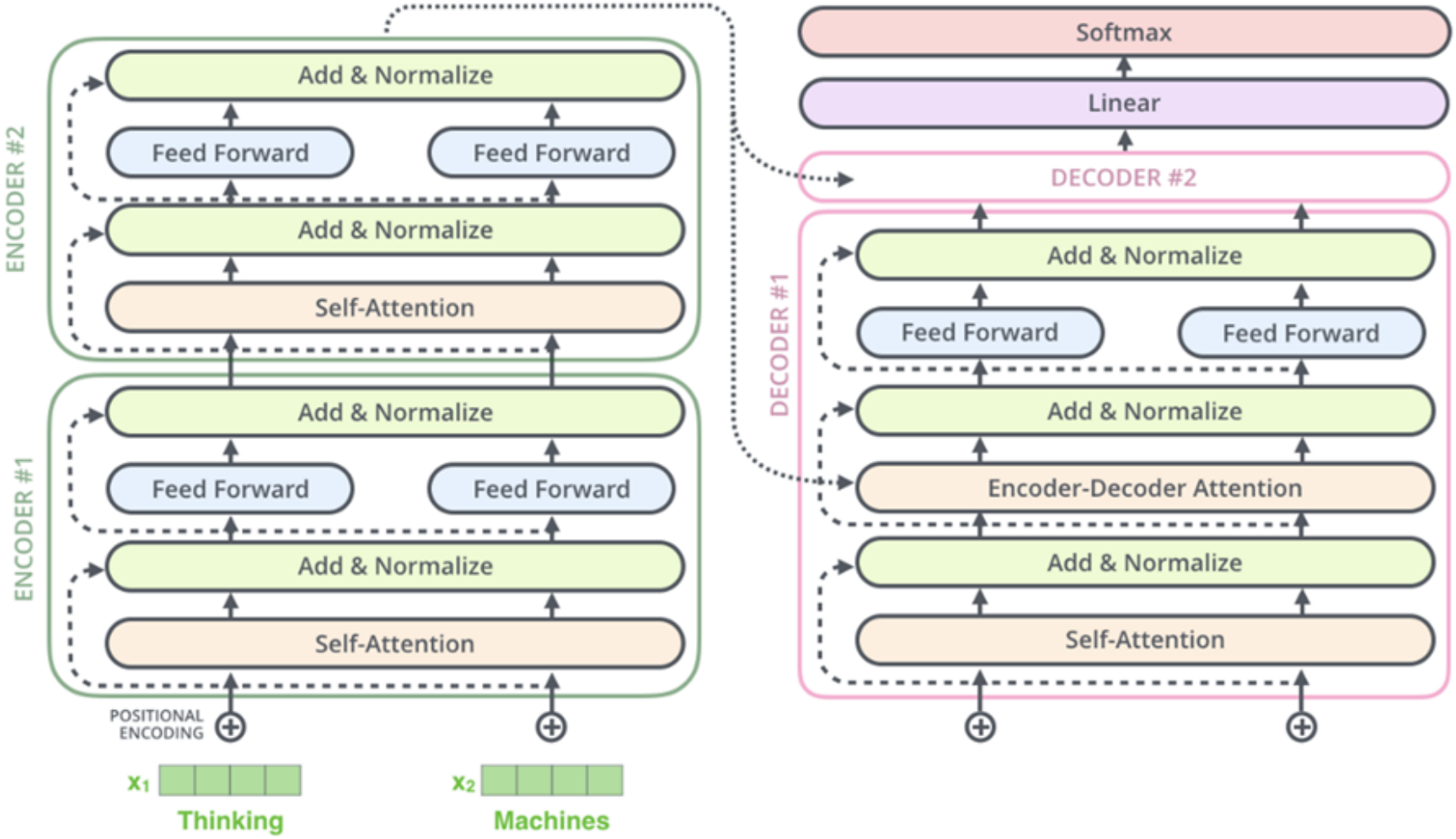
$$\mathbf{c}_t = \sum_k \sum_t a(W_k^Q \mathbf{x}_t, W_k^K \mathbf{z}_{t,out})\ W_k^V \mathbf{z}_{t,out}$$

- $\mathbf{x}_t$ is a layer in the decoder

- Positional Encoding: To address this, the transformer adds a vector to each input embedding. These vectors follow a specific pattern, which helps to determine the position of each word, or the distance between different words in the sequence.



Auto-Regressive: predicted word embeddings (all future inputs are masked)


ENCODER #2
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Self-Attention
ENCODER #1
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Self-Attention
POSITIONAL ENCODING
x1
x2
Thinking
Machines
Softmax
Linear
DECODER #2
DECODER #1
Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
Encoder-Decoder Attention
Add & Normalize
Self-Attention

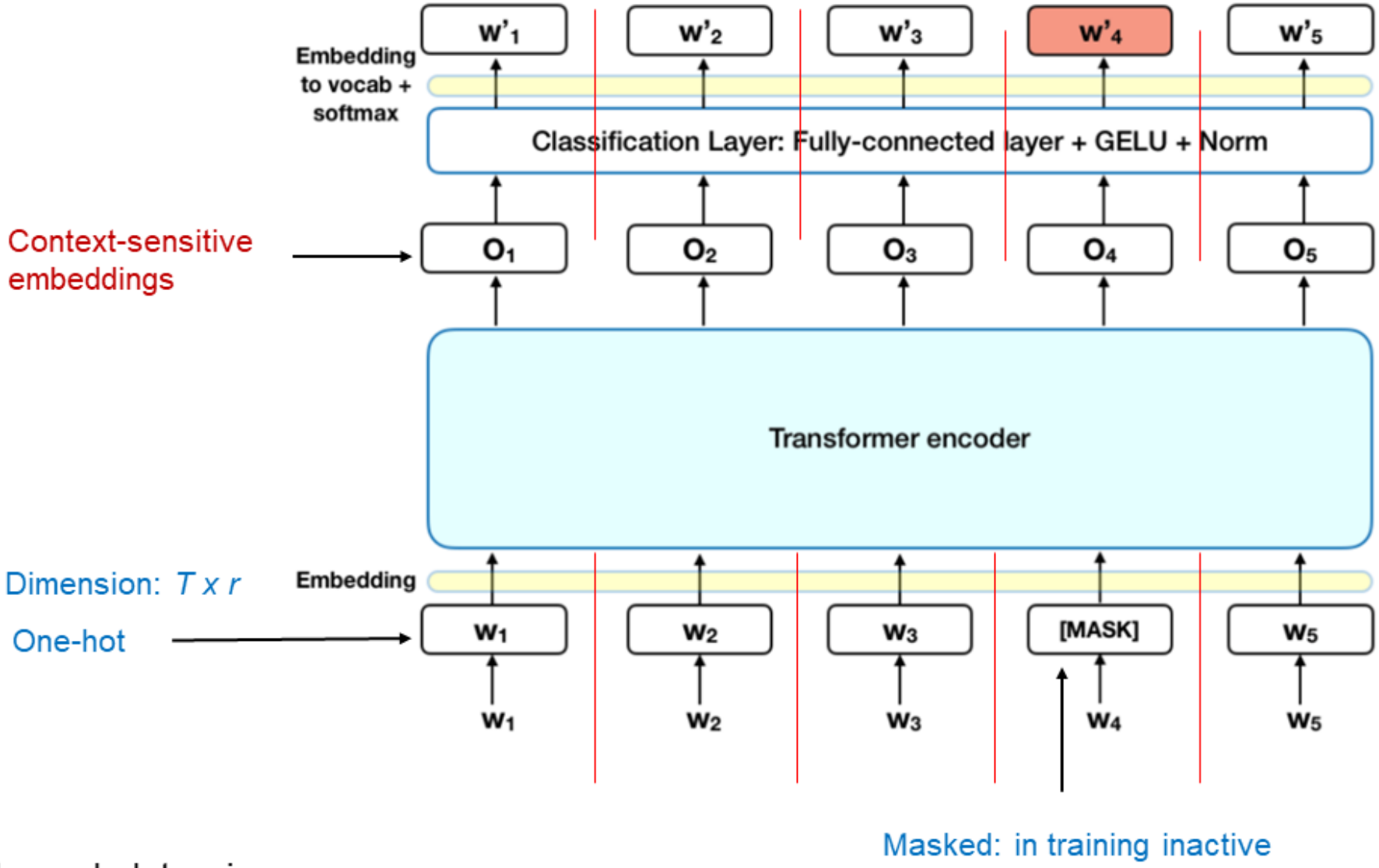
Decoder

- The decoder is auto-regressive
- It uses a one-step iterative greedy approach.
- At a higher "temperature" also less likely tokens are selected

BERT

- **BERT** (Bidirectional Encoder Representations from Transformers) from Google leverages attention mechanism and transformer to learn word contextual relations using a masked language model (MLM)
- ALBERT: A Lite BERT for Self-supervised Learning of Language Representations (Google)
- Longformer

BERT: Masked LM



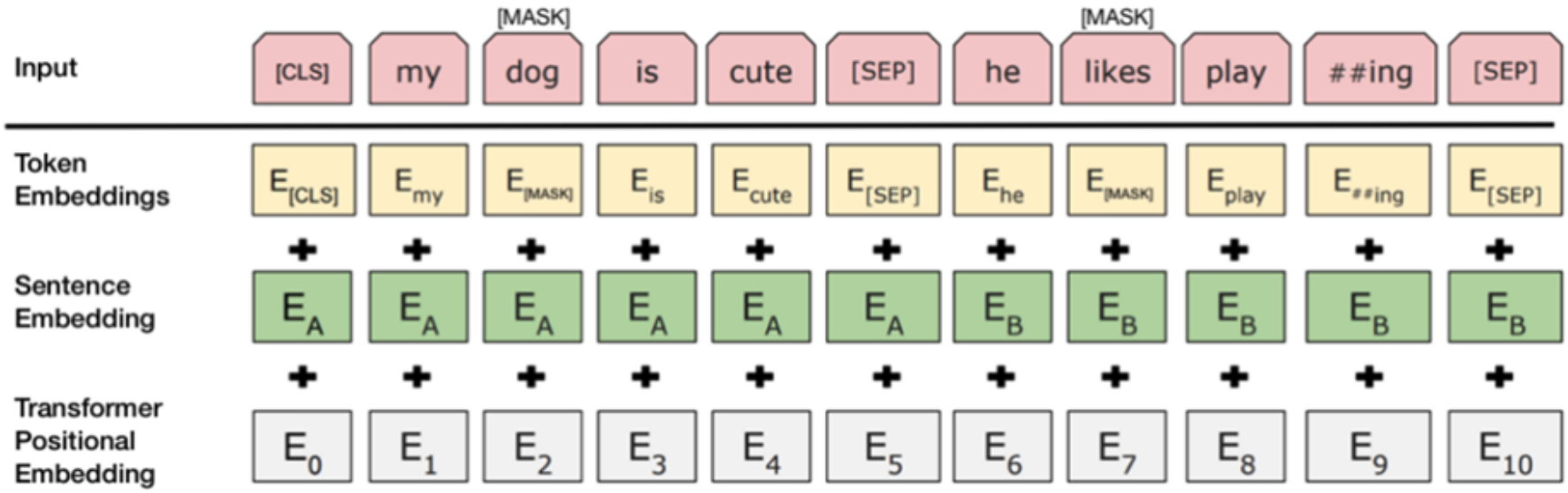
towardsdatascience

BERT

- BERT is almost an auto encoder: but some tokens of the input sentence are removed (masked) ad the network is trained to predict those tokens
- Masked language modelling (MLM)

BERT: Next Sentence Prediction

Also trained to predict: IsNextSequence or not



Source: BERT [Devlin et al., 2018], with modifications

BERT

- BERT performs next sentence prediction (NSP) but is not considered generative AI; In the BERT training process, the model receives pairs of sentences as input and learns to predict if the second sentence in the pair is the subsequent sentence in the original document.
- The actual embedding is a sum of the token embedding, the posidtion embedding and an embedding for first sentence or second sentence
- A sentence embedding indicating Sentence A or Sentence B is added to each token. Sentence embeddings are similar in concept to token embeddings with a vocabulary of 2

GPT

- Generative Pre-trained Transformer (GPT)
- Typically it only uses the Decoder part of the transformer
- It is initialized with the prompt (prompt engineering)
- It can handle sequences of 32k tokens
- GPT-1 to GPT-3: main difference are: different forms of training schedules!
- Attention is calculated w.r.t. all prompt tokens and all previously generated tokens

GPT

word, but also to continue on writing whole paragraphs in the style of the initial prompt. For example, when I gave GPT-3 the prompt:

In 1958, Frank Rosenblatt introduced the Perceptron, an early version of a neural net that could only detect the location of a square on a printed card. Fifty years later, neural nets are capable of . . .

The software dutifully completed the thought, and then continued on, picking up on the historical framing of the initial text:

. . . detecting and classifying faces, finding patterns in data, and emulating the human brain. While the neural net matured in academia, it also found its way into the tech industry. In the late 1980s and early 90s, neural nets were used in speech recognition and character recognition applications. The growth in computing power and the ability to collect larger data sets allowed other neural net applications to emerge.

New York Times, 2022-04-22

Foundation Models: A New Age?

- Different phases in machine learning:
- Before 2012: Small scale, feature-based learning
- From 2012 on: Deep Learning: large models with large sets of labelled data, supervised
- From 2020/2023 on: Foundation Models: large models with large sets of unlabelled data, self-supervised / unsupervised